

URCap Software Development Tutorial Swing

Universal Robots A/S

Version 1.18.0

Abstract

URCaps make it possible to seamlessly extend any Universal Robot with customized functionality. Using the URCap Software Platform, a URCap developer can define customized installation screens and program nodes for the end user. These can, for example, encapsulate new complex robot programming concepts, or provide friendly hardware configuration interfaces.

This tutorial explains how to use the URCap Software Platform version 1.18.0 to develop and deploy URCaps with Swing-based user interfaces for PolyScope version 5.23.0 running on e-Series robots. Older PolyScope versions can be targeted using older versions of the URCap Software Platform. URCaps can be developed for CB3 robots with PolyScope version 3.3.x to 3.15.x using version 1.12.0 or older of the URCap Software Platform.

Contents

1	Introduction	3
1.1	Features in URCap Software Platform 1.18.0	3
2	Prerequisites	5
3	URCap SDK	6
4	Building and deploying URCaps	8
4.1	Building	8
4.2	Manual Deployment using PolyScope	8
5	Structure of a URCap Project	13
6	Deployment with Maven	16
7	Contribution of an Installation Node	17
7.1	UI of the Installation Node View	17
7.2	Making the customized Installation Node available to PolyScope	19
7.3	Functionality of the Installation Node	20
7.4	Life Cycle of Contributions and Views	22
8	Contribution of a Program Node	23
8.1	UI of the Program Node View	23
8.2	Linking View and Contribution	25
8.3	Making the customized Program Nodes available to PolyScope	25
8.4	Functionality of the Program Node	27
8.5	Updating the data model	30
8.5.1	Restrictions	30
8.5.2	Undo/redo Functionality	30
8.6	Loading Programs with Program Node Contributions	31
8.7	Life Cycle of Contributions and Views	31
9	Contribution of a Daemon	33
9.1	Daemon Service	33
9.2	Interaction with the Daemon	34
9.3	C/C++ Daemon Executables	36
9.4	Tying the different Contributions together	37
10	URCap Examples Overview	39
10.1	Regular Samples	39
10.2	Driver Samples	45
11	Creating new thin Projects using a Maven Archetype	50
12	Compatibility	53
12.1	Robot Series Compatibility	53
12.1.1	Compatibility Flags	53
12.1.2	Installation and Startup of URCaps	54
12.1.3	Updating Existing URCaps	54
12.2	API Compatibility	55
12.2.1	Advanced compatibility	56

13 Exception Handling	58
14 Troubleshooting	59
A URCaps and Generated Script Code	61
B My Daemon Program and Installation Node	62

1 Introduction

The first official version of the URCap Software Platform (version 1.0.0) was released with PolyScope version 3.3.0 running on CB3 robots. This tutorial describes features supported in version 1.18.0 of the URCap Software Platform which is released together with PolyScope version 5.23.0 for e-Series robots. To create URCaps for PolyScope version 3.3.x to 3.15.x running on CB3 robots, older versions of the URCap Software Platform must be used.

This platform can be used to develop external contributions to PolyScope that are loaded when PolyScope starts up. This makes it possible for a URCap developer to provide customized functionality within PolyScope.

For example, a customized installation screen can serve the end user to comfortably configure a new tool. Similarly, a customized program node may serve as a way to perform complex tasks while hiding unnecessary detail.

The layout of a customized screen, the behaviour of a customized node, data persistence and script code generation is all implemented in Java. The URCap along with its resources is packaged and distributed as a single Java Jar file with the `.urcap` file extension. A URCap can be installed from the URCap Settings/Setup screen in PolyScope.

The tutorial is organized in the following manner:

- Section [2. Prerequisites](#) and [3. URCap SDK](#) explain what you need to start developing URCaps.
- Section [4. Building and deploying URCaps](#) to [6. Deployment with Maven](#) guide you through the basic project setup including build and deployment.
- Section [7. Contribution of an Installation Node](#) to [9. Contribution of a Daemon](#) introduces the concept behind URCaps and explains the different software components.
- Section [10. URCap Examples Overview](#) provides an overview of technical URCap examples distributed with the SDK that focus on specific features of the URCap API.
- Section [11. Creating new thin Projects using a Maven Archetype](#) demonstrates how to create an empty URCap project. We recommend that you also have a look at the examples when you want to start from scratch.
- Section [14. Troubleshooting](#) describes different debugging and troubleshooting options. Also visit the support forum at www.universal-robots.com/plus.

To get started we use the *Hello World Swing* and the *My Daemon Swing* URCaps as running examples. These are very simple and basic URCaps.

1.1 Features in URCap Software Platform 1.18.0

The following entities can be contributed to PolyScope using a URCap:

- Customized installation nodes and corresponding screens
- Customized program nodes and corresponding screens
- Daemon executables that run as separate background processes on the control box.

The customized installation nodes support:

- Saving and loading of data underlying the customized installation node as part of the currently loaded installation in PolyScope.
- Script code that an installation node contributes to the preamble of a robot program.

The customized program nodes support:

- Saving and loading of data underlying the customized program nodes as part of the currently loaded PolyScope program.
- Script code that a program node contributes to the script code generated by the robot program.

2 Prerequisites

A working version of Java SDK 6 is required for URCap development along with Apache Maven 3.0.5. You will also need PolyScope version 5.23.0 in order to install the developed URCap, if it is using URCap API version 1.18.0. Previous versions of the API will have lower requirements for the PolyScope version. Any Universal Robots robot (e.g., a UR3, UR5, UR10, etc.) can be used for that purpose or the Universal Robots offline simulator software (URSim). PolyScope and the offline simulator can be found in the download area of the tech support website at:

`www.universal-robots.com/support`

Select the applicable version and follow the given installation instructions. The offline simulator is available for Linux and non-Linux operating systems through a virtual Linux machine.

The script language and pre-defined script functions are defined in the script manual, which can also be found in the download area of the tech support website.

The URCap SDK is freely available on the Universal Robots+ website at:

`https://plus.universal-robots.com`

It includes the sources for the URCap examples.

The *My Daemon Swing* example of this tutorial additionally requires either Python 2.5 (compatible) or the Universal Robots urtool3 cross-compiler toolchain. The urtool3 cross-compiler is included in the SDK.

The following section [3. URCap SDK](#) describes the content of the URCap SDK.

3 URCap SDK

The URCap SDK provides the basics to create a URCap. It contains a Java package with the API that the developer will program against, documentation, the Hello World Swing and other URCap examples, the urtool3 cross-compiler toolchain and means of easily creating a new empty Maven-based template URCap project (See section [11. Creating new thin Projects using a Maven Archetype](#)) as well as easily updating an existing URCap.

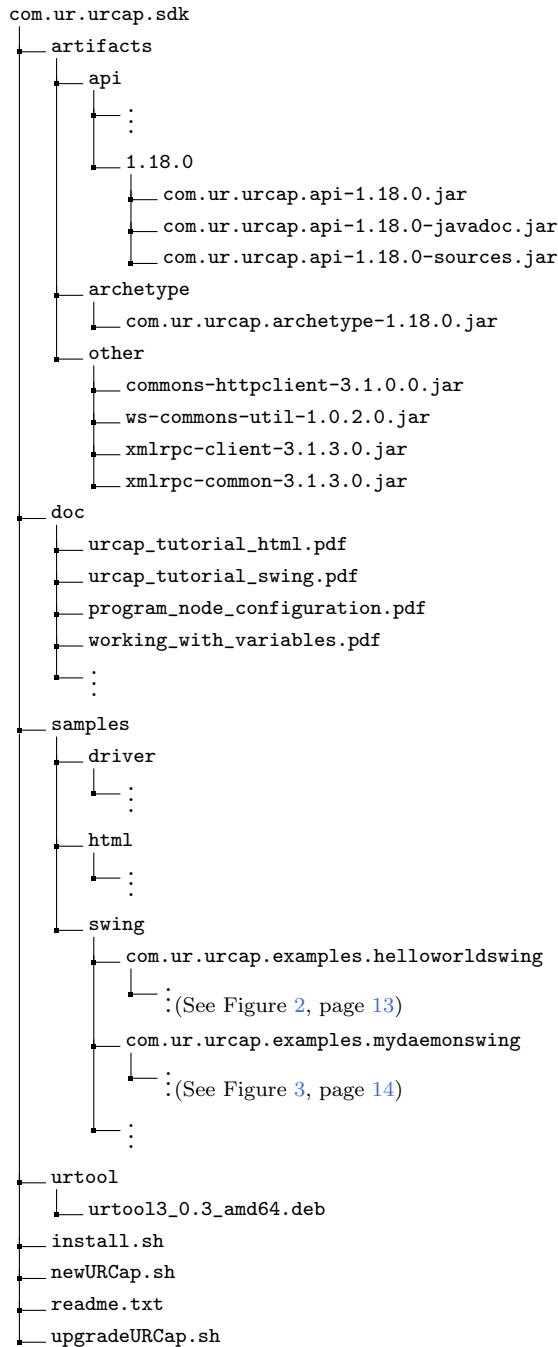


Figure 1: File structure of the URCaps SDK

The URCap SDK is distributed as a single ZIP file. Figure 1 shows the structure of the file.

A description of the directories and files contained in this file is given below:

/artifacts/: This directory holds all released versions of the URCap API in separate folders, the Maven archetype and other necessary files. Each folder named with a version number holds the Java packages, (e.g. `com.ur.urcap.api-1.18.0*.jar` files), that contains Java interfaces, Javadoc and sources of the URCap API that are necessary to implement the Java portion of a URCap. The Maven archetype folder holds the `com.ur.urcap.archetype-1.18.0.jar` file, that can be used to create a new empty template URCap project.

/doc/: The directory contains this tutorial (both in a HTML-based version and a Swing-based version) as well as a document describing how to configure child program nodes in a subtree and a document explaining how to work with variables. Included is also a document with a guide on how to convert an existing URCap with HTML-based user interface to a Swing-based one.

/samples/: A folder containing example projects demonstrating different features of the software framework. A description of the examples is found in section 10. [URCap Examples Overview](#).

/urtool/: Contains the urtool3 cross-compiler toolchain that should be used when building C/C++ daemon executables for the CB3.0/3.1 and CB5.0 control boxes.

install.sh: A script which should be run as a first step to install the URCap SDK and urtool3 cross-compiler toolchain (see section 4. [Building and deploying URCaps](#)). This will install all released versions of the URCap API and the Maven archetype in your local Maven repository as well as the cross-compiler toolchain in `/opt/urtool-3.0` (should you choose so).

newURCap.sh: A script which can be used to create a new empty Maven-based template URCap project in the current working directory (see section 11. [Creating new thin Projects using a Maven Archetype](#)).

upgradeURCap.sh: A script which can be used to update an existing URCap (project) to specify which robot series the URCap is compatible with (see section 12.1. [Robot Series Compatibility](#)).

readme.txt: A *readme* file describing the content of the SDK.

4 Building and deploying URCaps

4.1 Building

To get started unzip the SDK zip file to a suitable location and run the install script inside the target location:

```
1 $ ./install.sh
```

This installs the SDK on your machine.

Next, enter the `samples/swing/com.ur.urcap.examples.helloworldswing` directory and compile the example by:

```
1 $ cd samples/swing/com.ur.urcap.examples.helloworldswing
2 $ mvn install
```

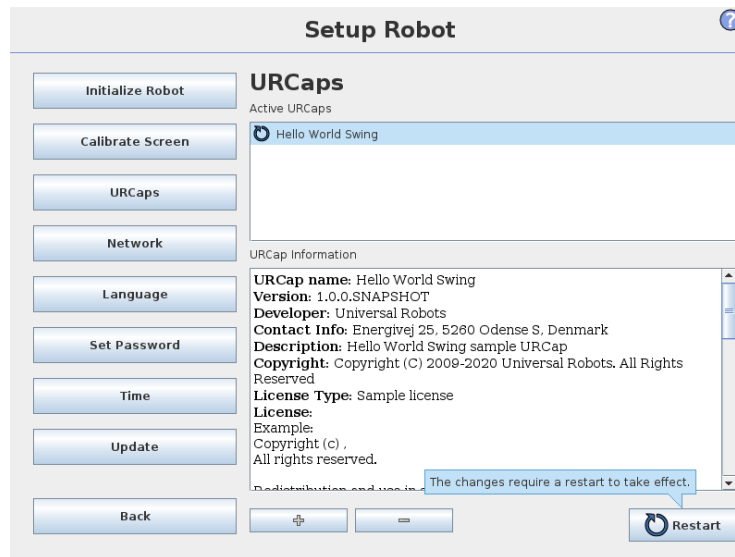
A new URCap with file name `target/helloworldswing-1.0-SNAPSHOT.urcap` has been born!

A similar procedure should be followed to compile the other URCap examples.

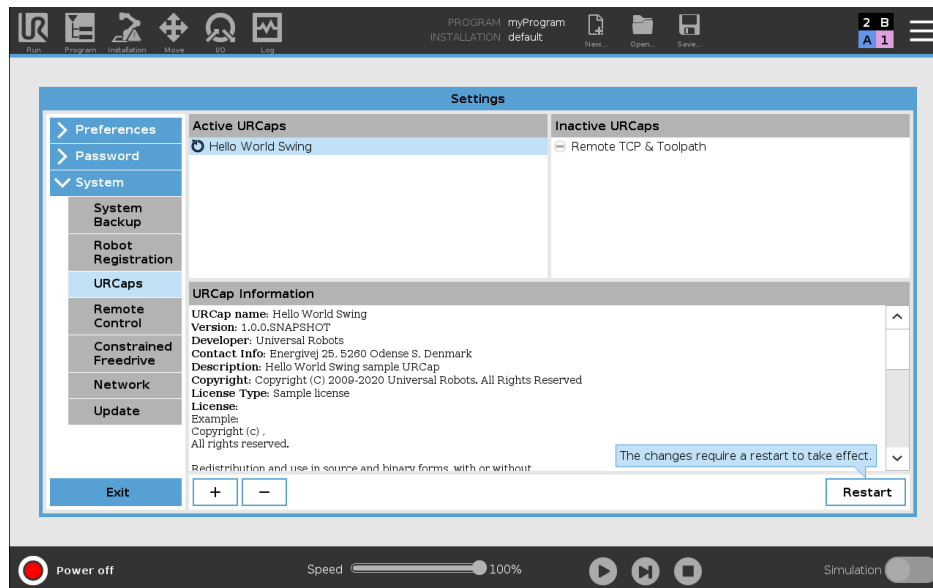
4.2 Manual Deployment using PolyScope

The URCap can be added to PolyScope with these steps:

1. Copy the `helloworldswing-1.0-SNAPSHOT.urcap` file from above to your `programs` directory used by PolyScope or to a USB stick and insert it into a robot.
2. **CB3 robot:** Tap the Setup Robot button from the PolyScope Robot User Interface Screen (the Welcome screen)
e-Series robot: In the right corner of the Header, tap the Hamburger menu and select Settings
3. **CB3 Robot:** Tap the URCaps button from the Setup Robot screen.
e-Series robot: In the Settings screen under System, select URCaps.
4. Tap the + button.
5. Select a `.urcap` file, e.g. `helloworldswing-1.0-SNAPSHOT.urcap` and tap the Open button.
6. Restart PolyScope using the Restart button in the bottom of the screen:

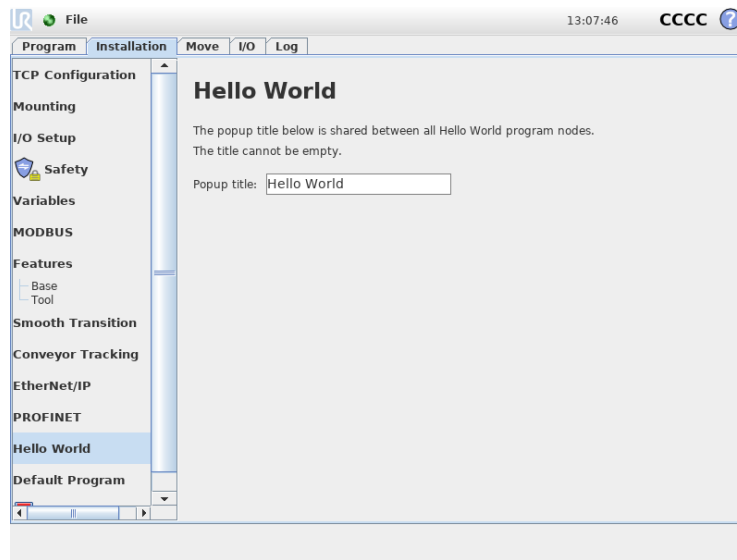


CB3 robot

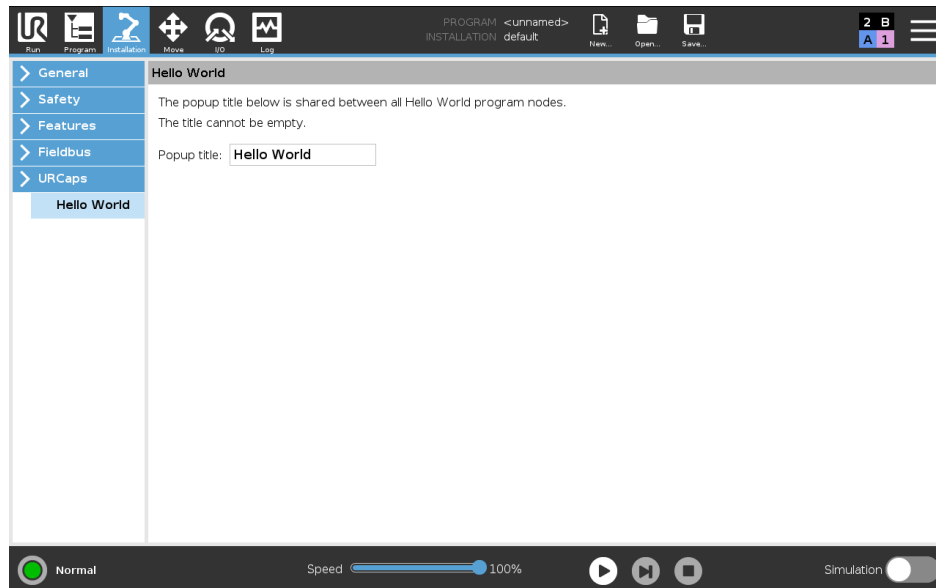


e-Series robot

When the Hello World Swing URCap is deployed, the following installation screen is accessible from the Installation tab:

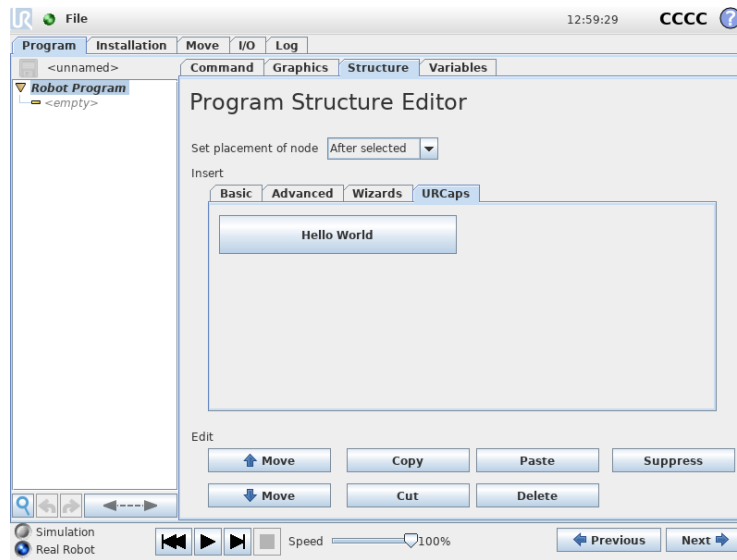


CB3 robot



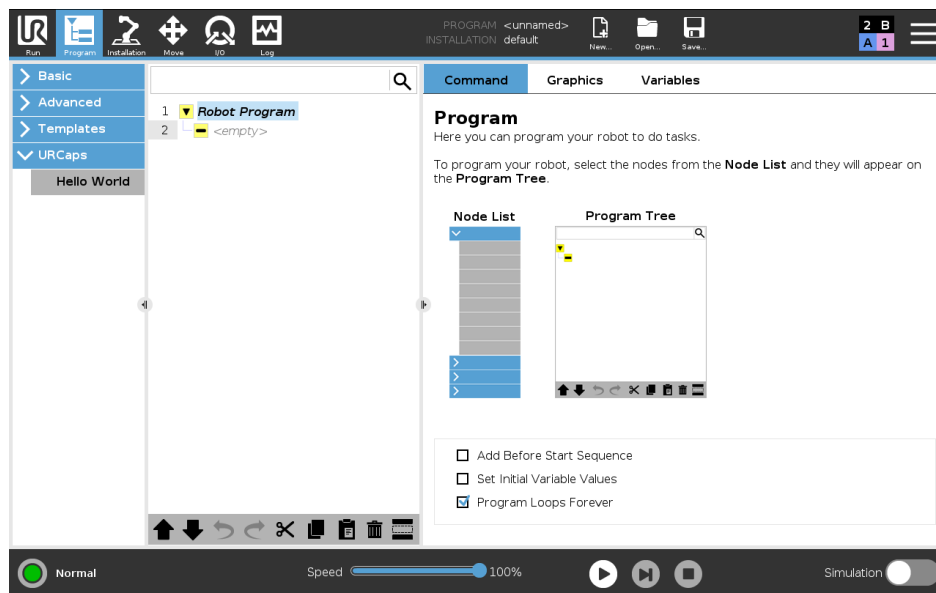
e-Series robot

Furthermore, the Hello World program node is available in the Program tab. On CB3 robots, the node is visible within the Structure tab after selecting the URCaps subtab:



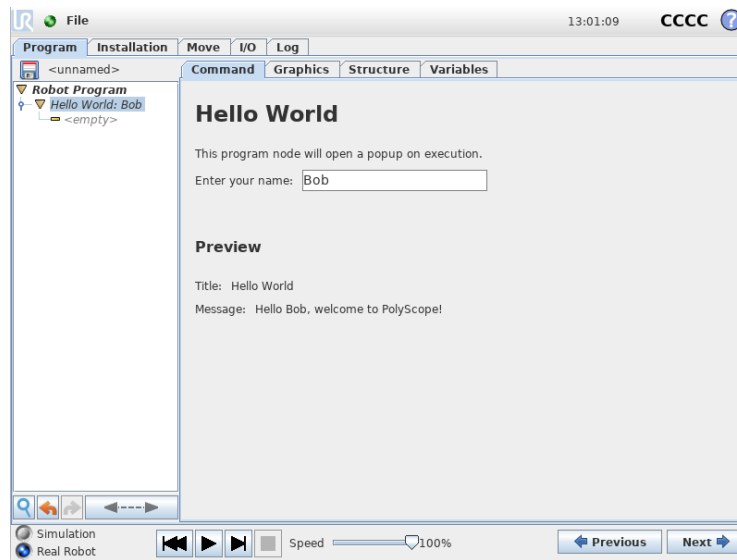
CB3 robot

On e-Series robots, the node is visible on the left of the screen after tapping URCaps:

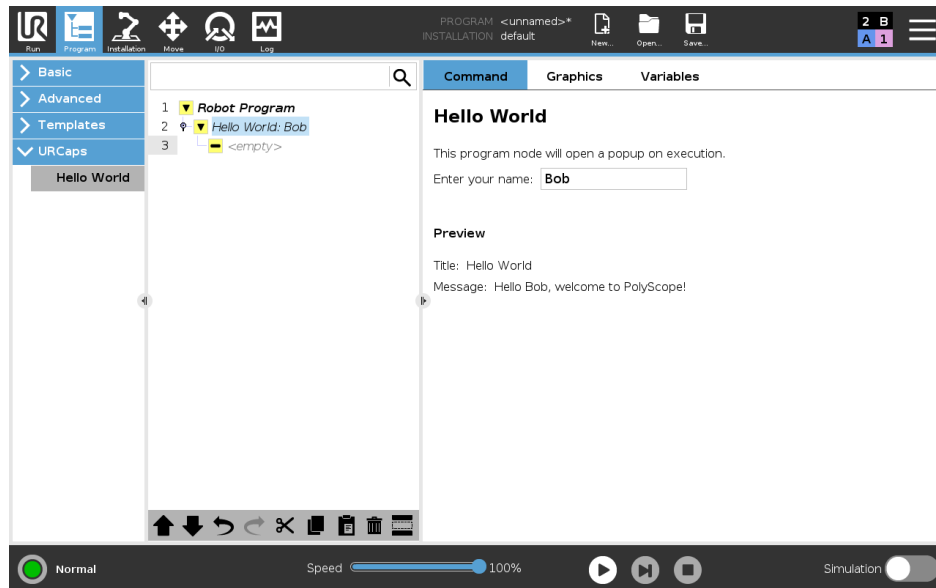


e-Series robot

The screen for the program node looks as follows:



CB3 robot



e-Series robot

When the program displayed above runs, a pop-up is shown with the title "Hello World" (configured in the installation screen) and message "Hello Bob, welcome to PolyScope!" (using the name defined in the program node).

5 Structure of a URCap Project

A URCap is a Java Archive (.jar) file with the .urcap file extension. The Java file may contain a number of new installation nodes, program nodes, and daemon executables.

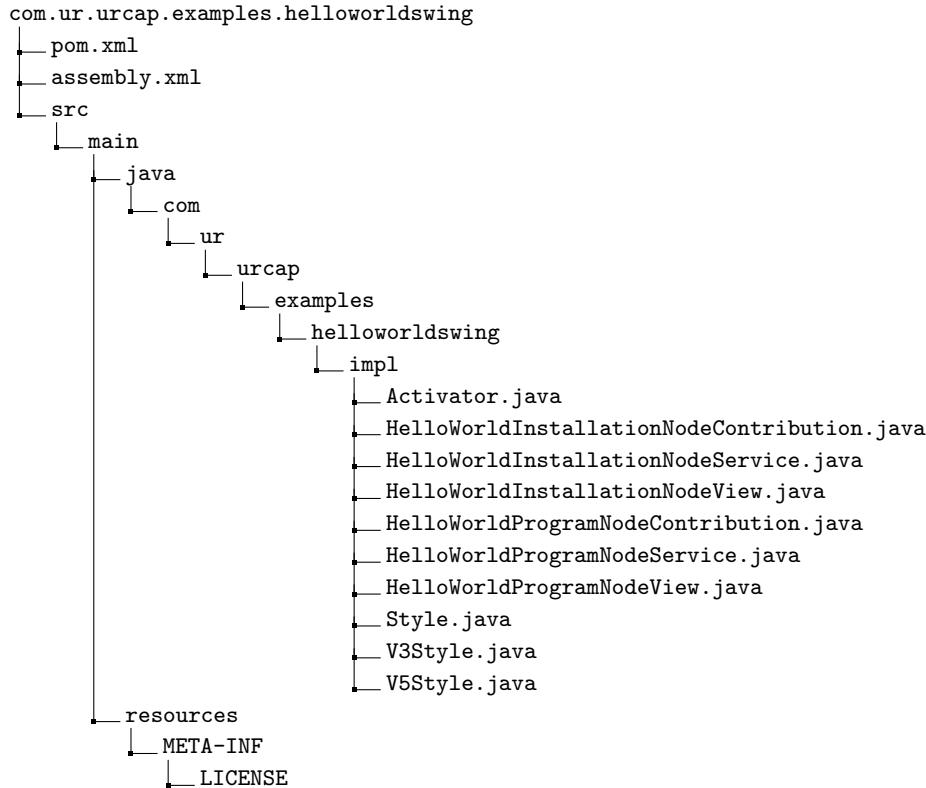


Figure 2: Structure of the Hello World Swing URCap project

Figure 2 shows the structure of the Hello World Swing URCap project. This project consists of the following parts:

1. A Java *view* part consisting of two screens with the layout specified in the two files `HelloWorldProgramNodeView.java` and `HelloWorldInstallationNodeView.java`.
2. A Java *implementation* for the screens above, namely:
 - (a) `HelloWorldInstallationNodeService.java` and `HelloWorldInstallationNodeContribution.java`
 - (b) `HelloWorldProgramNodeService.java` and `HelloWorldProgramNodeContribution.java`
3. A *license* `META-INF/LICENSE` with the license information that is shown to the user when the URCap is installed.
4. Maven configuration files `pom.xml` and `assembly.xml` for building the project.

The My Daemon Swing URCap is an extended version of the Hello World Swing URCap, that exemplifies the integration of an external daemon process. Figure 3 shows the structure of the My Daemon Swing URCap project. Compared to the Hello World Swing project it additionally offers the following parts:

1. A Python 2.5 *daemon* executable in the file `hello-world.py`.
2. C++ *daemon* sources in directory `daemon`.
3. A Java *implementation* `MyDaemonDaemonService.java` that defines and installs a daemon and makes it possible to control the daemon.

The Python and C++ daemons are alternatives that provide the same functionality.

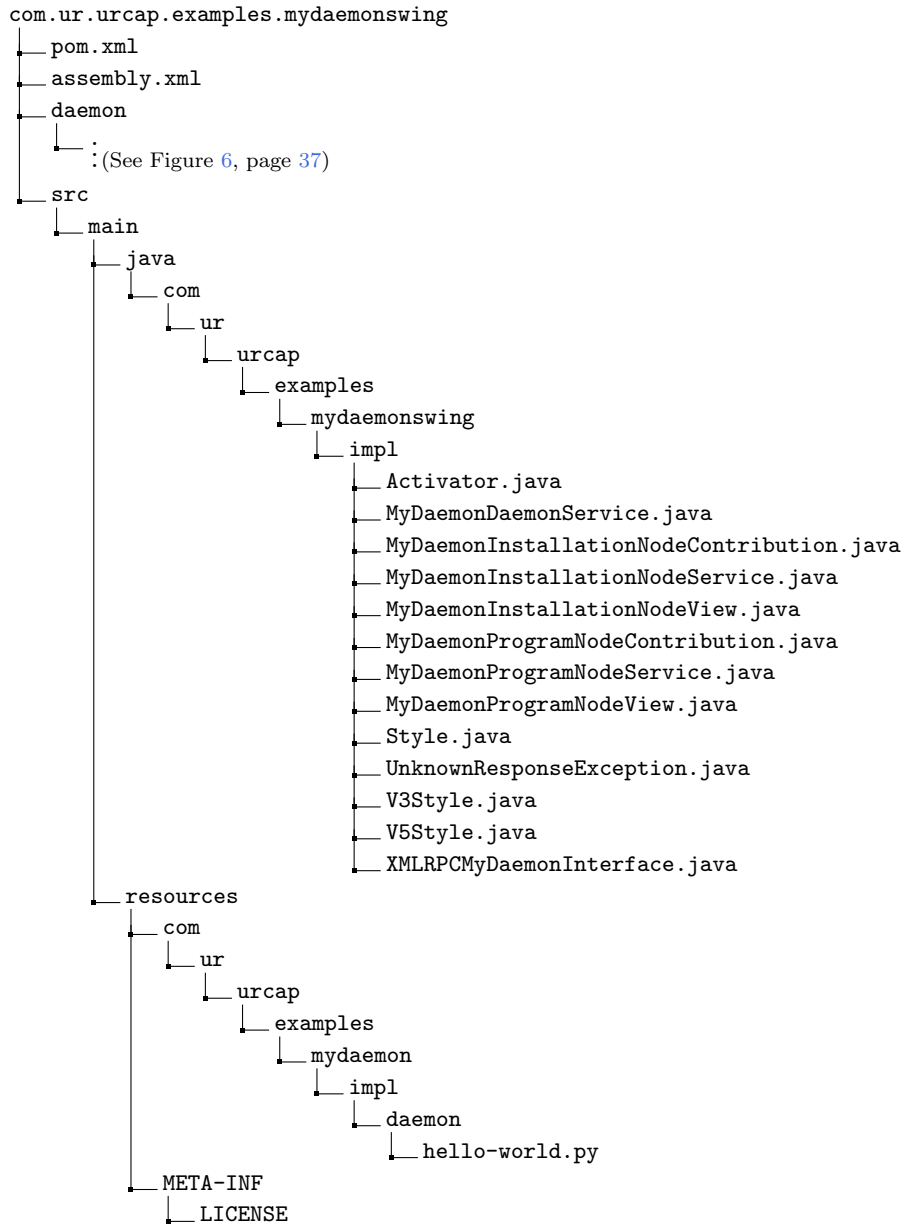


Figure 3: Structure of the My Daemon Swing URCap project

The services:

- HelloWorldInstallationNodeService.java
- HelloWorldProgramNodeService.java

are registered in `Activator.java` and thereby a new installation node and program node are offered to PolyScope. The My Daemon Swing additionally registers its `MyDaemonDaemonService.java` service to make the daemon executable available to PolyScope.

The file `pom.xml` contains a subsection (under the `<properties>` section) with a set of properties for the URCap with meta-data specifying the vendor, contact address, copyright, description, and short license information which will be displayed to the user when the URCap is installed in PolyScope. See Listing 1 for the Hello World Swing version of these properties.

Listing 1: Section with meta-data properties inside the `pom.xml` file for the Hello World Swing URCap

```

1  <!--*****-->
2  <!--    Note: Update this section with relevant meta data    -->
3  <!--          that comes along with your URCap          -->
4  <!--*****-->
5  <!--***** BEGINNING OF URCAP META DATA *****-->
6  <urcap.symbolicname>com.ur.urcap.examples.helloworldswing</urcap.symbolicname>
7  <urcap.vendor>Universal Robots</urcap.vendor>
8  <urcap.contactAddress>Energivej 25, 5260 Odense S, Denmark</urcap.
    contactAddress>
9  <urcap.copyright>Copyright (C) 2009-2025 Universal Robots. All Rights Reserved
    </urcap.copyright>
10 <urcap.description>Hello World Swing sample URCap</urcap.description>
11 <urcap.licenseType>Sample license</urcap.licenseType>
12 <urcap.compatibility.CB3>true</urcap.compatibility.CB3>
13 <urcap.compatibility.eSeries>true</urcap.compatibility.eSeries>
14 <!--***** END OF URCAP META DATA *****-->

```

The URCap meta-data subsection also contains two boolean compatibility flag properties (`urcap.compatibility.CB3` and `urcap.compatibility.eSeries`) which specify the URCap's compatibility with the CB3 and the e-Series robot series, respectively. See section [12.1. Robot Series Compatibility](#) for more information about these compatibility flags.

6 Deployment with Maven

In order to ease development, a URCap can be deployed using Maven.

Deployment to a robot with Maven Given the IP address of the robot, e.g. 10.2.128.64, go to your URCap project folder and type:

```
1 $ cd samples/swing/com.ur.urcap.examples.helloworldswing
2 $ mvn install -Premote -Durcap.install.host=10.2.128.64
```

and the URCap is deployed and installed on the robot. During this process PolyScope will be restarted.

You can also specify the IP address of the robot via the property `urcap.install.host` inside the `pom.xml` file. Then you can deploy by typing:

```
1 $ cd samples/swing/com.ur.urcap.examples.helloworldswing
2 $ mvn install -Premote
```

Deployment to URSim If you are running Linux then URSim can be installed locally. Otherwise it needs to run in a virtual machine (VM). It is possible to deploy to both environments with Maven. As shown above parameters can be supplied either directly on the command line or in the `pom.xml` file.

- To deploy to a *locally running URSim* specify the path to the extracted URSim with the property `ursim.home`.
- To deploy to a *URSim running in a VM* specify the IP address of the VM using the property `ursimvm.install.host`.

Once the properties are configured you can deploy to a local URSim by using the `ursim` profile:

```
1 $ cd samples/swing/com.ur.urcap.examples.helloworldswing
2 $ mvn install -P ursim
```

or the URSim running in a VM using the `ursimvm` profile:

```
1 $ cd samples/swing/com.ur.urcap.examples.helloworldswing
2 $ mvn install -P ursimvm
```

Note, if you are using VirtualBox to run the VM you should make sure that the network of the VM is operating in bridged mode.

7 Contribution of an Installation Node

A URCap can contribute installation nodes. An installation node will support a customized installation node screen and customized functionality.

7.1 UI of the Installation Node View

The layout of a customized installation node screen is defined by a Java class implementing the `SwingInstallationNodeView` interface where the Swing GUI framework is used to create the user interface (UI). The implementation must specify the associated installation node contribution implementing the installation node's functionality (described in section 7.3. [Functionality of the Installation Node](#)) as a type variable.

Listing 2: The view (UI) of the customized Hello World installation screen

```

1  package com.ur.urcap.examples.helloworldswing.impl;
2
3  import com.ur.urcap.api.contribution.installation.swing.
    SwingInstallationNodeView;
4  import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardTextInput;
5
6  import javax.swing.BorderFactory;
7  import javax.swing.Box;
8  import javax.swing.BoxLayout;
9  import javax.swing.JLabel;
10 import javax.swing.JPanel;
11 import javax.swing.JTextField;
12 import javax.swing.JTextPane;
13 import javax.swing.text.SimpleAttributeSet;
14 import javax.swing.text.StyleConstants;
15 import java.awt.Component;
16 import java.awt.Dimension;
17 import java.awt.event.MouseAdapter;
18 import java.awt.event.MouseEvent;
19
20 public class HelloWorldInstallationNodeView implements
    SwingInstallationNodeView<HelloWorldInstallationNodeContribution> {
21
22     private final Style style;
23     private JTextField jTextField;
24
25     public HelloWorldInstallationNodeView(Style style) {
26         this.style = style;
27     }
28
29     @Override
30     public void buildUI(JPanel jPanel, final
        HelloWorldInstallationNodeContribution installationNode) {
31         jPanel.setLayout(new BoxLayout(jPanel, BoxLayout.Y_AXIS));
32
33         jPanel.add(createInfo());
34         jPanel.add(createVerticalSpacing());
35         jPanel.add(createInput(installationNode));
36     }
37
38     private Box createInfo() {
39         Box infoBox = Box.createVerticalBox();
40         infoBox.setAlignmentX(Component.LEFT_ALIGNMENT);
41         JTextPane pane = new JTextPane();
42         pane.setBorder(BorderFactory.createEmptyBorder());
43         SimpleAttributeSet attributeSet = new SimpleAttributeSet();
44         StyleConstants.setLineSpacing(attributeSet, 0.5f);
45         StyleConstants.setLeftIndent(attributeSet, 0f);
46         pane.setParagraphAttributes(attributeSet, false);

```

```

47     pane.setText("The popup title below is shared between all Hello World
        program nodes.\n\nThe title cannot be empty.");
48     pane.setEditable(false);
49     pane.setMaximumSize(pane.getPreferredSize());
50     pane.setBackground(infoBox.getBackground());
51     infoBox.add(pane);
52     return infoBox;
53 }
54
55 private Box createInput(final HelloWorldInstallationNodeContribution
        installationNode) {
56     Box inputBox = Box.createHorizontalBox();
57     inputBox.setAlignmentX(Component.LEFT_ALIGNMENT);
58
59     inputBox.add(new JLabel("Popup title:"));
60     inputBox.add(createHorizontalSpacing());
61
62     jTextField = new JTextField();
63     jTextField.setFocusable(false);
64     jTextField.setPreferredSize(style.getInputfieldSize());
65     jTextField.setMaximumSize(jTextField.getPreferredSize());
66     jTextField.addMouseListener(new MouseAdapter() {
67         @Override
68         public void mousePressed(MouseEvent e) {
69             KeyboardTextInput keyboardInput = installationNode.
                getInputForTextField();
70             keyboardInput.show(jTextField, installationNode.
                getCallbackForTextField());
71         }
72     });
73     inputBox.add(jTextField);
74
75     return inputBox;
76 }
77
78 private Component createHorizontalSpacing() {
79     return Box.createRigidArea(new Dimension(style.getHorizontalSpacing(), 0));
80 }
81
82 private Component createVerticalSpacing() {
83     return Box.createRigidArea(new Dimension(0, style.getVerticalSpacing()));
84 }
85
86 public void setPopupText(String t) {
87     jTextField.setText(t);
88 }
89 }

```

Listing 2 shows the content of the `HelloWorldInstallationNodeView.java` file which defines the layout of the screen used for the Hello World installation node. The class uses various Swing GUI components to construct the user interface. These components are added to the `JPanel` provided as argument in the `buildUI(JPanel, InstallationNodeContribution)` method.

The panel has a fixed size that cannot be changed. Margins are already added by PolyScope, so the entire area of the panel can be used for UI components. The title will also be set automatically to the value returned by `SwingInstallationNodeService.getTitle(Locale)` (described in section 7.2. [Making the customized Installation Node available to PolyScope](#)). It will have a Swing UI manager already set, meaning that components without additional styling will look as native PolyScope ones including the correct font types. In order to resemble PolyScope the components should therefore only use limited styling, such as font sizes or input field sizes (if the applied ones are not suitable). Calling methods not supported by PolyScope will result in

an exception being thrown.

The corresponding installation node contribution is passed as the second argument to the method `buildUI(JPanel, InstallationNodeContribution)` to enable the view and the contribution to communicate with each other in order to pass values and react to events.

This structure creates a model-view separation where the view is created in the aforementioned class and the model is handled in the contribution. The corresponding Java code is presented in the following two sections.

7.2 Making the customized Installation Node available to PolyScope

In order to make the layout specified in the view class and the customized installation nodes available to PolyScope, a Java class that implements the interface `SwingInstallationNodeService` must be defined. Listing 3 shows the Java code that makes the Hello World installation node available to PolyScope.

Listing 3: Hello World Installation node service

```

1  package com.ur.urcap.examples.helloworldswing.impl;
2
3  import com.ur.urcap.api.contribution.ViewAPIProvider;
4  import com.ur.urcap.api.contribution.installation.ContributionConfiguration;
5  import com.ur.urcap.api.contribution.installation.CreationContext;
6  import com.ur.urcap.api.contribution.installation.InstallationAPIProvider;
7  import com.ur.urcap.api.contribution.installation.swing.
      SwingInstallationNodeService;
8  import com.ur.urcap.api.domain.SystemAPI;
9  import com.ur.urcap.api.domain.data.DataModel;
10
11 import java.util.Locale;
12
13 public class HelloWorldInstallationNodeService implements
      SwingInstallationNodeService<HelloWorldInstallationNodeContribution,
      HelloWorldInstallationNodeView> {
14
15     @Override
16     public void configureContribution(ContributionConfiguration configuration) {
17     }
18
19     @Override
20     public String getTitle(Locale locale) {
21         return "Hello World";
22     }
23
24     @Override
25     public HelloWorldInstallationNodeView createView(ViewAPIProvider apiProvider
        ) {
26         SystemAPI systemAPI = apiProvider.getSystemAPI();
27         Style style = systemAPI.getSoftwareVersion().getMajorVersion() >= 5 ? new
            V5Style() : new V3Style();
28         return new HelloWorldInstallationNodeView(style);
29     }
30
31     @Override
32     public HelloWorldInstallationNodeContribution createInstallationNode(
        InstallationAPIProvider apiProvider, HelloWorldInstallationNodeView view
        , DataModel model, CreationContext context) {
33         return new HelloWorldInstallationNodeContribution(apiProvider, model, view
            );
34     }
35 }

```

The `SwingInstallationNodeService` interface requires two type variables for the specific implementations of the `InstallationNodeContribution` and `SwingInstallationNodeView` interface, respectively, as well as the following methods to be defined:

- `getTitle(Locale)` returns the title for the node, to be shown on the left side of the Installation tab to access the customized installation screen. For simplicity, the title is specified simply as `"HelloWorld"`. In a more realistic example, the return value of the `getTitle(Locale)` method would be translated into the language specified by standard Java localization, based on the provided `Locale` argument. The title is also used as the fixed header for the installation node screen in PolyScope. The method is only called once.
- `createView(ViewAPIProvider)` returns an instance of the view (described in section 7.1. [UI of the Installation Node View](#)). Use the `ViewAPIProvider` to access information about the software version, language settings (can be used to provide a translated UI), etc.
- `createInstallationNode(InstallationAPIProvider, SwingInstallationNodeView, DataModel, CreationContext)` is called by PolyScope when it needs to create an instance of the installation node. The arguments are:
 - `InstallationAPIProvider`: provides access to various PolyScope domain APIs relevant for an installation node
 - `SwingInstallationNodeView`: the view instance created by the `createView(ViewAPIProvider)` described above
 - `DataModel`: the model for the installation node with automatic persistence support
 - `CreationContext`: the context in which the `createInstallationNode(...)` is called

The constructor used in the implementation of the method `createInstallationNode(...)` is discussed in section 7.3. [Functionality of the Installation Node](#). All modifications to the supplied data model from the installation node constructor are ignored when existing installation is loaded. This means that ideally the installation node constructor should not set anything in the data model.

- `configureContribution(ContributionConfiguration)` is called once after the service has been registered. Use the argument supplied to configure the contribution if its default values are not appropriate (see default values in the Javadoc). If the default values are appropriate, leave this method empty.

7.3 Functionality of the Installation Node

The functionality behind a customized installation node must be defined in a Java class that implements the `InstallationNodeContribution` interface. Listing 4 shows the Java code that defines the functionality of the Hello World installation screen. An instance of this class is returned by the `createInstallationNode(...)` method in the `HelloWorldInstallationNodeService` class described in previous section.

In essence, the `InstallationNodeContribution` interface requires the following to be defined:

1. What happens when the user enters and exits the customized installation screen.
2. Script code that should be added to the preamble of any program when run with this URCap installed.

In addition, the class contains code that links to the view (mentioned in section 7.1. [UI of the Installation Node View](#)), gives access to a data model with automatic persistence and UR-Script generation associated with the node.

Listing 4: Java class defining functionality for the Hello World installation node

```

1 package com.ur.urcap.examples.helloworldswing.impl;
2
3 import com.ur.urcap.api.contribution.InstallationNodeContribution;
4 import com.ur.urcap.api.contribution.installation.InstallationAPIProvider;
5 import com.ur.urcap.api.domain.data.DataModel;
6 import com.ur.urcap.api.domain.script.ScriptWriter;
7 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputCallback;
8 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputFactory;
9 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardTextInput;
10
11 public class HelloWorldInstallationNodeContribution implements
    InstallationNodeContribution {
12
13     private static final String POPUPTITLE_KEY = "popuptitle";
14     private static final String DEFAULT_VALUE = "Hello World";
15     private final HelloWorldInstallationNodeView view;
16     private final KeyboardInputFactory keyboardFactory;
17
18     private DataModel model;
19
20     public HelloWorldInstallationNodeContribution(InstallationAPIProvider
        apiProvider, DataModel model, HelloWorldInstallationNodeView view) {
21         this.keyboardFactory = apiProvider.getUserInterfaceAPI().
            getUserInteraction().getKeyboardInputFactory();
22         this.model = model;
23         this.view = view;
24     }
25
26     @Override
27     public void openView() {
28         view.setPopupText(getPopupTitle());
29     }
30
31     @Override
32     public void closeView() {
33
34     }
35
36     public boolean isDefined() {
37         return !getPopupTitle().isEmpty();
38     }
39
40     @Override
41     public void generateScript(ScriptWriter writer) {
42         // Store the popup title in a global variable, so it is globally available
43         // to all Hello World program nodes.
44         writer.assign("hello_world_swing_popup_title", "\"" + getPopupTitle() + "
45         \");
46
47     }
48
49     public String getPopupTitle() {
50         return model.get(POPUPTITLE_KEY, DEFAULT_VALUE);
51     }
52
53     public void setPopupTitle(String message) {
54         if (!"".equals(message)) {
55             resetToDefaultValue();
56         } else {
57             model.set(POPUPTITLE_KEY, message);
58         }
59     }
60
61     private void resetToDefaultValue() {
62         view.setPopupText(DEFAULT_VALUE);
63         model.set(POPUPTITLE_KEY, DEFAULT_VALUE);
64     }
65
66 }

```

```

61     }
62
63     public KeyboardTextInput getInputForTextField() {
64         KeyboardTextInput keyboardInput = keyboardFactory.
65             createStringKeyboardInput();
66         keyboardInput.setInitialValue(getPopupTitle());
67         return keyboardInput;
68     }
69
70     public KeyboardInputCallback<String> getCallbackForTextField() {
71         return new KeyboardInputCallback<String>() {
72             @Override
73             public void onOk(String value) {
74                 setPopupTitle(value);
75                 view.setPopupText(value);
76             }
77         };
78     }

```

The data model which was mentioned in section 7.2. [Making the customized Installation Node available to PolyScope](#) is passed into the constructor through a `DataModel` object. All data that needs to be saved and loaded along with a robot installation must be stored in and retrieved from this model object.

When the user interacts with the text input field defined in the view, the listener defined there will request a keyboard from the contribution and when the user accepts, delegate the call to `getCallbackForTextField()`. The code within that method takes care of storing the contents of the text input widget in the data model under the key `POPUPTITLE_KEY` whenever the user accepts what is typed using the keyboard. By saving and loading the robot installation you will notice that values are stored and read again from and back to the view.

The `openView()` method is called whenever the user enters the screen. It sets the contents of the text input field defined in the view to the value stored in the data model. The `closeView()` method is called when the user leaves the screen.

Finally, the preamble of each program run with this URcap installed will contain an assignment in its preamble, as specified in the implementation of the `generateScript(ScriptWriter)` method. In the assignment, the script variable named `"hello_world_swing_popup_title"` is assigned to a string that contains the popup title stored within the data model object.

7.4 Life Cycle of Contributions and Views

Each time a new installation is created or a different installation is loaded, the `createView(...)` method in the `SwingInstallationNodeService` interface (see section 7.2. [Making the customized Installation Node available to PolyScope](#)) is also called and a new view instance should be returned. The `createInstallationNode(...)` method in the interface `SwingInstallationNodeService` is also called to pass in the new `DataModel` instance.

This means that Java garbage collection has a chance to clean up previous instances in case any listeners have been forgotten or other potential memory leaks have been created. This also means that no references to the view instance or contribution instance should be kept outside these classes. Respecting this will protect PolyScope from running out of memory.

8 Contribution of a Program Node

A URCap can contribute program nodes. A node is supplied by a customized view part and a part with the customized functionality.

8.1 UI of the Program Node View

The layout for customized program node screens is defined similarly as the layout of customized installation node screens (see section 7.1. [UI of the Installation Node View](#)) by implementing the `SwingProgramNodeView` interface. The implementation must specify the associated program node contribution implementing the functionality of the program nodes (described in section 8.4. [Functionality of the Program Node](#)) as a type variable. The panel provided to the `buildUI(JPanel, ContributionProvider<ProgramNodeContribution>)` method has the same restrictions and properties as described in section 7.1. [UI of the Installation Node View](#).

Listing 5 shows the definition of the layout of a simple program node. It contains a single input text field where the user can type a name and two labels that provide a preview of the popup that will be displayed at runtime. A label defined in the view will be used to display the title set in the installation. The name that is entered by the end user in the Hello World program node will be used to construct a customized popup message. This message will also be shown in the preview label with name "previewMessage".

In order to communicate with the corresponding program node contribution implementing the functionality of the program node itself, a provider is passed as argument to the method `buildUI(JPanel, ContributionProvider<ProgramNodeContribution>)`. Calling the `get()` method on the `ContributionProvider` object will return the currently selected program node. The provider has a type variable which corresponds to the associated contribution.

The linking of the view and the program node contributions is presented in the following section.

Listing 5: The view (UI) of the customized Hello World program screen

```

1 package com.ur.urcap.examples.helloworldswing.impl;
2
3 import com.ur.urcap.api.contribution.ContributionProvider;
4 import com.ur.urcap.api.contribution.program.swing.SwingProgramNodeView;
5 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardTextInput;
6
7 import javax.swing.Box;
8 import javax.swing.BoxLayout;
9 import javax.swing.JLabel;
10 import javax.swing.JPanel;
11 import javax.swing.JTextField;
12 import java.awt.Component;
13 import java.awt.Dimension;
14 import java.awt.Font;
15 import java.awt.event.MouseAdapter;
16 import java.awt.event.MouseEvent;
17
18 public class HelloWorldProgramNodeView implements SwingProgramNodeView<
19     HelloWorldProgramNodeContribution>{
20     private final Style style;
21     private JTextField jTextField;
22     private JLabel previewTitle;
23     private JLabel previewMessage;
24
25     public HelloWorldProgramNodeView(Style style) {
26         this.style = style;
27     }

```



```

28
29  @Override
30  public void buildUI(JPanel jPanel, final ContributionProvider<
    HelloWorldProgramNodeContribution> provider) {
31      jPanel.setLayout(new BoxLayout(jPanel, BoxLayout.Y_AXIS));
32
33      jPanel.add(createInfo());
34      jPanel.add(createVerticalSpacing(style.getVerticalSpacing()));
35      jPanel.add(createInput(provider));
36      jPanel.add(createVerticalSpacing(style.getExtraLargeVerticalSpacing()));
37      jPanel.add(createPreview());
38  }
39
40  private Box createInfo() {
41      Box infoBox = Box.createHorizontalBox();
42      infoBox.setAlignmentX(Component.LEFT_ALIGNMENT);
43      infoBox.add(new JLabel("This program node will open a popup on execution."
44          ));
45      return infoBox;
46  }
47
48  private Box createInput(final ContributionProvider<
    HelloWorldProgramNodeContribution> provider) {
49      Box inputBox = Box.createHorizontalBox();
50      inputBox.setAlignmentX(Component.LEFT_ALIGNMENT);
51      inputBox.add(new JLabel("Enter your name:"));
52      inputBox.add(createHorizontalSpacing());
53
54      JTextField = new JTextField();
55      JTextField.setFocusable(false);
56      JTextField.setPreferredSize(style.getInputfieldSize());
57      JTextField.setMaximumSize(JTextField.getPreferredSize());
58      JTextField.addMouseListener(new MouseAdapter() {
59          @Override
60          public void mousePressed(MouseEvent e) {
61              KeyboardTextInput keyboardInput = provider.get().
62                  getKeyboardForTextField();
63              keyboardInput.show(JTextField, provider.get().getCallbackForTextField
64                  ());
65          }
66      });
67
68      inputBox.add(JTextField);
69      return inputBox;
70  }
71
72  private Box createPreview() {
73      Box previewBox = Box.createVerticalBox();
74      JLabel preview = new JLabel("Preview");
75      preview.setFont(preview.getFont().deriveFont(Font.BOLD, style.
76          getSmallHeaderFontSize()));
77
78      Box titleBox = Box.createHorizontalBox();
79      titleBox.setAlignmentX(Component.LEFT_ALIGNMENT);
80      titleBox.add(new JLabel("Title:"));
81      titleBox.add(createHorizontalSpacing());
82      previewTitle = new JLabel("my title");
83      titleBox.add(previewTitle);
84
85      Box messageBox = Box.createHorizontalBox();
86      messageBox.setAlignmentX(Component.LEFT_ALIGNMENT);
87      messageBox.add(new JLabel("Message:"));
88      messageBox.add(createHorizontalSpacing());
89      previewMessage = new JLabel("my message");
90      messageBox.add(previewMessage);
91
92      previewBox.add(preview);

```

```

89     previewBox.add(createVerticalSpacing(style.getLargeVerticalSpacing()));
90     previewBox.add(titleBox);
91     previewBox.add(createVerticalSpacing(style.getVerticalSpacing()));
92     previewBox.add(messageBox);
93
94     return previewBox;
95 }
96
97 private Component createVerticalSpacing(int height) {
98     return Box.createRigidArea(new Dimension(0, height));
99 }
100
101 private Component createHorizontalSpacing() {
102     return Box.createRigidArea(new Dimension(style.getHorizontalSpacing(), 0));
103 }
104
105 public void setPopupText(String popupText) {
106     jTextField.setText(popupText);
107 }
108
109 public void setMessagePreview(String message) {
110     previewMessage.setText(message);
111 }
112
113 public void setTitlePreview(String title) {
114     previewTitle.setText(title);
115 }
116 }

```

8.2 Linking View and Contribution

The view (implementing the `SwingProgramNodeView` interface) and the program node contribution (implementing the `ProgramNodeContribution` interface) must be able to communicate in order to pass values and react to events. Only one view instance exists and many instances of the contribution could exist.

In order for the view to call methods on the currently selected program node's underlying contribution, the supplied provider must be used. The `get()` method on the `ContributionProvider` will automatically return the `ProgramNodeContribution` instance representing the currently selected program node and in turn its associated data model.

The contribution object on the other hand was instantiated with the one and only view instance and can call methods on this instance without any further ado.

8.3 Making the customized Program Nodes available to PolyScope

To make the Hello World program node available to PolyScope, a Java class that implements the `SwingProgramNodeService` interface is required. Listing 6 shows the Java code that makes the Hello World program node available to PolyScope.

The `getId()` method returns the unique identifier for this type of program node. The identifier will be used when storing programs that contain these program nodes. This method is called once. Do not change the return value of this method in released URCaps, since it will break backwards compatibility for existing programs. URCap program nodes in such existing programs will not be loaded properly and the program can not run anymore.

Its `getTitle(Locale)` method supplies the text for the button in the Structure Tab that corresponds to this type of program node. It is also used as the title on the Command tab screen

for such nodes. Use the provided `Locale` if translated titles should be supported. This method is called once.

Use the argument supplied in the method `configureContribution(ContributionConfiguration)` to configure the program node contribution. This method is called once after the service has been registered. Use the argument supplied to configure the contribution if its default values are not appropriate (see default values in the Javadoc). If the default values are appropriate, leave this method empty. The following properties can be configured:

- Calling `setDeprecated()` with `true` makes it impossible to create new program nodes of this type, but still support loading program nodes of this type in existing programs.
- Calling `setChildrenAllowed()` with `true`, signals that it is possible for the program node to contain other (child) program nodes.
- Calling `setUserInsertable()` with `false` makes the program node programmatically insertable only (i.e. the end user cannot insert it).
- Calling `getProgramDebuggingSupport()` returns `ProgramDebuggingSupport` object that can be used to customize debugging capabilities of the Program Node and child nodes in subtree. Debugging features are available on e-Series platform only. Refer to User Manual for detailed description.
 - `setAllowBreakpointOnNode()` determines if the end user is allowed to set a breakpoint on or single step this program node.
 - `setAllowBreakpointOnChildNodesInSubtree()` determines if the end user is allowed to set breakpoint on or single step any child in this program node subtree (only if the child node itself allows breakpoints).
 - `setAllowStartFromNode()` determines if the end user can start the program directly from this program node.
 - `setAllowStartFromChildNodesInSubtree()` determines if the end user can start the program directly from the selected child in this program node subtree (only if the child node itself allows it).

Finally, `createNode(ProgramAPIProvider, SwingProgramNodeView, DataModel, CreationContext)` creates program nodes. The arguments are:

- `ProgramAPIProvider`: provides access to various APIs relevant for a program node.
- `SwingProgramNodeView`: this is the view instance (described in section 8.1. UI of the Program Node View) created by the `createView(ViewAPIProvider)` method
- `DataModel`: this gives the user a data model with automatic persistence
- `CreationContext`: the context in which this program node is created

The `createNode(...)` method creates a new Java object for each node of this type occurring in the program tree. The returned object is used when interacting with the view on the customized program node screen for the particular node selected in the program tree. It must use the supplied data model object to retrieve and store data that should be saved and loaded within the robot program along with the corresponding node occurrence. Please note that only data related to the current configuration of that particular program node instance should be stored in the data model, i.e. no global or shared state, state irrelevant to this node instance, etc. should be stored.

The constructor used in the implementation of the `createNode(...)` method is discussed in section 8.4. Functionality of the Program Node. The `createNode(...)` method call during program load is discussed in section 8.6. Loading Programs with Program Node Contributions.

Listing 6: Java class defining how Hello World program nodes are created

```

1 package com.ur.urcap.examples.helloworldswing.impl;
2
3 import com.ur.urcap.api.contribution.ViewAPIProvider;
4 import com.ur.urcap.api.contribution.program.ContributionConfiguration;
5 import com.ur.urcap.api.contribution.program.CreationContext;
6 import com.ur.urcap.api.contribution.program.ProgramAPIProvider;
7 import com.ur.urcap.api.contribution.program.swing.SwingProgramNodeService;
8 import com.ur.urcap.api.domain.SystemAPI;
9 import com.ur.urcap.api.domain.data.DataModel;
10
11 import java.util.Locale;
12
13 public class HelloWorldProgramNodeService implements SwingProgramNodeService<
14     HelloWorldProgramNodeContribution, HelloWorldProgramNodeView> {
15
16     @Override
17     public String getId() {
18         return "HelloWorldSwingNode";
19     }
20
21     @Override
22     public void configureContribution(ContributionConfiguration configuration) {
23         configuration.setChildrenAllowed(true);
24     }
25
26     @Override
27     public String getTitle(Locale locale) {
28         String title = "Hello World";
29         if ("ru".equals(locale.getLanguage())) {
30             title = "Привет";
31         } else if ("de".equals(locale.getLanguage())) {
32             title = "Hallo Welt";
33         }
34         return title;
35     }
36
37     @Override
38     public HelloWorldProgramNodeView createView(ViewAPIProvider apiProvider) {
39         SystemAPI systemAPI = apiProvider.getSystemAPI();
40         Style style = systemAPI.getSoftwareVersion().getMajorVersion() >= 5 ? new
41             V5Style() : new V3Style();
42         return new HelloWorldProgramNodeView(style);
43     }
44
45     @Override
46     public HelloWorldProgramNodeContribution createNode(
47         ProgramAPIProvider apiProvider,
48         HelloWorldProgramNodeView view,
49         DataModel model,
50         CreationContext context) {
51         return new HelloWorldProgramNodeContribution(apiProvider, view, model);
52     }
53 }

```

8.4 Functionality of the Program Node

The functionality of the Hello World program node is implemented in the Java class shown in Listing 7. This class implements the `ProgramNodeContribution` interface and instances of this class are returned by the method `createNode(ProgramAPIProvider, SwingProgramNodeView, DataModel, CreationContext)` of the `HelloWorldProgramNodeService` class described in the previous section.

Listing 7: Java class defining functionality for the Hello World program node

```

1  package com.ur.urcap.examples.helloworldswing.impl;
2
3  import com.ur.urcap.api.contribution.ProgramNodeContribution;
4  import com.ur.urcap.api.contribution.program.ProgramAPIProvider;
5  import com.ur.urcap.api.domain.ProgramAPI;
6  import com.ur.urcap.api.domain.data.DataModel;
7  import com.ur.urcap.api.domain.script.ScriptWriter;
8  import com.ur.urcap.api.domain.undoredo.UndoRedoManager;
9  import com.ur.urcap.api.domain.undoredo.UndoableChanges;
10 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputCallback;
11 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputFactory;
12 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardTextInput;
13
14 public class HelloWorldProgramNodeContribution implements
15     ProgramNodeContribution {
16     private static final String NAME = "name";
17
18     private final ProgramAPI programAPI;
19     private final UndoRedoManager undoRedoManager;
20     private final KeyboardInputFactory keyboardFactory;
21
22     private final HelloWorldProgramNodeView view;
23     private final DataModel model;
24
25     public HelloWorldProgramNodeContribution(ProgramAPIProvider apiProvider,
26         HelloWorldProgramNodeView view, DataModel model) {
27         this.programAPI = apiProvider.getProgramAPI();
28         this.undoRedoManager = apiProvider.getProgramAPI().getUndoRedoManager();
29         this.keyboardFactory = apiProvider.getUserInterfaceAPI().
30             getUserInteraction().getKeyboardInputFactory();
31
32         this.view = view;
33         this.model = model;
34     }
35
36     @Override
37     public void openView() {
38         view.setPopupText(getName());
39         updatePopupMenuAndPreview();
40     }
41
42     @Override
43     public void closeView() {
44     }
45
46     @Override
47     public String getTitle() {
48         return "Hello World: " + (model.isSet(NAME) ? getName() : "");
49     }
50
51     @Override
52     public boolean isDefined() {
53         return getInstallation().isDefined() && !getName().isEmpty();
54     }
55
56     @Override
57     public void generateScript(ScriptWriter writer) {
58         // Directly generate this Program Node's popup message + access the popup
59         // title through a global variable
60         writer.appendLine("popup(\"" + generatePopupMenu() + "\",
61             hello_world_swing_popup_title, false, false, blocking=True)");
62         writer.writeChildren();
63     }
64
65     public KeyboardTextInput getKeyboardForTextField() {

```

```

61     KeyboardTextInput keyboardInput = keyboardFactory.
        createStringKeyboardInput();
62     keyboardInput.setInitialValue(getName());
63     return keyboardInput;
64 }
65
66 public KeyboardInputCallback<String> getCallbackForTextField() {
67     return new KeyboardInputCallback<String>() {
68         @Override
69         public void onOk(String value) {
70             setPopupTitle(value);
71             view.setPopupText(value);
72         }
73     };
74 }
75
76 public void setPopupTitle(final String value) {
77     undoRedoManager.recordChanges(new UndoableChanges() {
78         @Override
79         public void executeChanges() {
80             if ("".equals(value)) {
81                 model.remove(NAME);
82             } else {
83                 model.set(NAME, value);
84             }
85         }
86     });
87
88     updatePopupMessageAndPreview();
89 }
90
91 private String generatePopupMessage() {
92     return model.isSet(NAME) ? "Hello_" + getName() + ",_welcome_to_PolyScope!"
93         : "No_name_set";
94 }
95
96 private void updatePopupMessageAndPreview() {
97     view.setMessagePreview(generatePopupMessage());
98     view.setTitlePreview(getInstallation().isDefined() ? getInstallation().
99         getPopupTitle() : "No_title_set");
100 }
101
102 private String getName() {
103     return model.get(NAME, "");
104 }
105
106 private HelloWorldInstallationNodeContribution getInstallation() {
107     return programAPI.getInstallationNode(
108         HelloWorldInstallationNodeContribution.class);
109 }

```

The `openView()` and `closeView()` methods specify what happens when the user selects and unselects the underlying program node in the program tree.

The `getTitle()` method defines the text which is displayed in the program tree for the node. The text of the node in the program tree is updated when values are written to the `DataModel`.

The `isDefined()` method serves to identify whether the node is completely defined (green) or still undefined (yellow). Note that a node, which can contain other program nodes (see section 8.3. Making the customized Program Nodes available to PolyScope), remains undefined as long as it has a child that is undefined. The `isDefined()` method is called when values are writ-

ten to the `DataModel` to ensure that the program tree reflects the proper state of the program node.

Finally, `generateScript(ScriptWriter)` is called to add script code to the spot where the underlying node occurs in the robot program.

As the user interacts with the text input field, the constructed message is displayed on the screen using the view instance. Each Hello World node is defined (green) if both the Hello World installation node is defined and the name in the program node is non-empty. When executed, it shows a simple popup dialog with the title defined in the installation and the message constructed from the name.

The popup title is the value of the script variable `hello_world_swing_popup_title`. This variable is initialized by the script code contributed by the Hello World installation node. Thus, the script variable serves to pass data from the contributed installation node to the contributed program node. Another approach to pass information between these two objects is by directly requesting the installation object through the `ProgramAPI` interface. The Hello World program node utilizes this approach in its `updatePopupMessageAndPreview()` method.

8.5 Updating the data model

8.5.1 Restrictions

It is not allowed to modify the supplied data model in the implementation of any overridden method defined in the program node contribution interface (described in section 8.4. [Functionality of the Program Node](#)), which PolyScope calls. The data model should rather be updated due to a user-initiated event, e.g. a button click.

Furthermore, if the URCap contains an installation node contribution, the data model of the installation contribution not be modified from the program node contribution. If such modifications occur in the call to `openView()` while a program is running, PolyScope will automatically stop the program every time the URCap program node is encountered in the program tree, because the installation was changed. It is, however, allowed to read from the data model in the installation node contribution.

8.5.2 Undo/redo Functionality

All user-initiated data model or program tree changes must happen inside the scope of an `UndoableChanges` object using the `UndoRedoManager` interface to record the changes. Multiple changes to the model or program tree can happen inside a single `UndoableChanges` object and this means that all said changes will be undoable as a single action by the end user. Below in Listing 8 is a small code snippet demonstrating this.

Listing 8: Code snippet for undo/redo

```
1  UndoRedoManager manager = apiProvider.getProgramAPI().getUndoRedoManager();
2  manager.recordChanges(new UndoableChanges() {
3      @Override
4      public void executeChanges() {
5          model.set(NAME, "my_name");
6          insertChildNodes();
7      }
8  });
```

Only if the end user initiated the action (e.g. clicked a button) should the changes be recorded as an undoable action. Otherwise the end user will be able to undo something he did not do

and be confused as to what he is undoing.

Failing to record changes inside an `UndoableChanges` will throw a `IllegalStateException` exception.

As mentioned in section 8.3. [Making the customized Program Nodes available to PolyScope](#), remember to only store data related to the current configuration of the particular program node instance in the model, i.e. no global or shared state, state irrelevant to this node instance, etc. should be stored there.

Undoable actions only apply to a program nodes. Changes to a data model in an installation node do not have undo/redo support and will not throw a `IllegalStateException` exception.

When a user clicks undo, the previous values are restored in the data model as well as the program tree and a call to the `openView()` method if the program node is currently selected, for an opportunity to display the new values.

This also means that no values should be cached in member variables, but always retrieved from the data model, as there is no guarantee that things have not changed. Also keep in mind, that the user might not select the Command tab of the URCap, so there is no guaranteed call to `openView()`. This can be the case when loading a program that has already been setup.

8.6 Loading Programs with Program Node Contributions

Program node contributions contain a data model and an interface to manipulate the sub-tree (introduced in URCap API version 1.1.0).

When a contributable program node is created in PolyScope by the user, the data model object given to the `createNode(ProgramAPIProvider, SwingProgramNodeView, DataModel, CreationContext)` method is empty and the provided `CreationContext` object will also reflect this.

When a program is loaded, the method `createNode(...)` is called for each persisted program node contribution to re-create the program tree. In contrast to newly creating the program node, the data model now contains the data from the persisted node and the `CreationContext` object will reflect this situation also. All modifications to the data model from the program node constructor are ignored. This means that ideally the program node constructor should not set anything in the data model.

For creating sub-trees a program model can be used. In some of the URCap examples in Chapter 10. [URCap Examples Overview](#) it is demonstrated how a sub-tree can be generated programmatically. The program model provides the interface `TreeNode` to create and manipulate the sub-tree. When a contributable program node is created in PolyScope by the user, the tree node has no children. The program model can be requested through the `ProgramAPI` interface.

When a program is loaded each program node is deserialized on its own, this includes sub-trees previously created through the program model. Also now, the tree node requested through the `ProgramModel` is empty. The program node factory returned by `getProgramNodeFactory()` in the `ProgramModel` interface will return program nodes without any functionality. In particular, the method `createURCapProgramNode(Class<? extends URCapProgramNodeService>)` does not call the `createNode(...)` method in the specified service. Therefore, modifications are ignored during the `createNode(...)` call.

8.7 Life Cycle of Contributions and Views

Each time a new program is created or a different program is loaded, the `createView(...)` method in the `SwingProgramNodeService` interface (see section [8.3. Making the customized Program Nodes available to PolyScope](#)) is also called and a new view instance should be returned. The method `createNode(...)` in the `SwingProgramNodeService` interface is also called to pass in the new `DataModel` object.

This means that Java garbage collection has a chance to clean up previous instances in case any listeners have been forgotten or other potential memory leaks have been created. This also means that no references to the view instance or contribution instance should be kept outside these classes. Respecting this will protect PolyScope from running out of memory.

9 Contribution of a Daemon

A daemon can be any executable script or binary file that runs on the control box. The My Daemon Swing URCap serves as the running example for explaining this functionality and is an extension of the Hello World Swing example. The My Daemon Swing example offers the same functionality from the user's point of view as the Hello World Swing example.

However, the My Daemon Swing URCap performs its tasks through an executable, which acts as a sort of driver or server. The executable is implemented as Python 2.5 script and C++ binary. The executables communicate with the Java front-end and URScript executor through XML encoded Remote Procedure Calls (XML-RPC). Figure 3, page 14, shows the structure of the My Daemon Swing URCap project.

9.1 Daemon Service

A URCap can contribute any number of daemon executables through implementation of the `DaemonService` interface (see Listing 9):

- The `init(DaemonContribution)` method will be called by PolyScope with a `DaemonContribution` object which gives the URCap developer the control to install, start, stop, and query the state of the daemon. An example of how to integrate start, stop, and query a daemon will be discussed in Section 9.2. [Interaction with the Daemon](#).
- The `installResource(URL url)` method in the `DaemonContribution` interface takes an argument that points to the source inside the URCap Jar file (`.urcap` file). This path may point to a single executable daemon or a directory that contains a daemon and additional files (e.g. dynamic linked libraries or configuration files).
- The implementation of `getExecutable()` provides PolyScope with the path to the executable that will be started.

The `/etc/service` directory contains links to the URCap daemon executables currently running. If a daemon executable has a link present but is in fact not running, the `ERROR` state will be returned upon querying the daemon's state. The links to daemon executables follow the lifetime of the encapsulating URCap and will be removed when the URCap is removed. The initial state for a daemon is `STOPPED`, however if it is desired, auto-start can be achieved by calling `start()` in the `init(DaemonContribution)` method right after the daemon has had its resources installed.

Listing 9: The My Daemon Service

```

1 package com.ur.urcap.examples.mydaemonswing.impl;
2
3 import com.ur.urcap.api.contribution.DaemonContribution;
4 import com.ur.urcap.api.contribution.DaemonService;
5
6 import java.net.MalformedURLException;
7 import java.net.URL;
8
9
10 public class MyDaemonDaemonService implements DaemonService {
11
12     private DaemonContribution daemonContribution;
13
14     public MyDaemonDaemonService() {
15     }
16
17     @Override
18     public void init(DaemonContribution daemonContribution) {
19         this.daemonContribution = daemonContribution;

```

```

20     try {
21         daemonContribution.installResource(new URL("file:com/ur/urcap/examples/
                mydaemonswing/impl/daemon/"));
22     } catch (MalformedURLException e) { }
23 }
24
25 @Override
26 public URL getExecutable() {
27     try {
28         // Two equivalent example daemons are available:
29         return new URL("file:com/ur/urcap/examples/mydaemonswing/impl/daemon/
                hello-world.py"); // Python executable
30 //         return new URL("file:com/ur/urcap/examples/mydaemonswing/impl/daemon/
                HelloWorld"); // C++ executable
31     } catch (MalformedURLException e) {
32         return null;
33     }
34 }
35
36 public DaemonContribution getDaemon() {
37     return daemonContribution;
38 }
39
40 }

```

Log information with respect to the process handling of the daemon executable are saved together with the daemon executable (follow the symbolic link of the daemon executable in `/etc/service` to locate the log directory).

Note, that script daemons must include an interpreter directive at the first line to help select the right program for interpreting the script. For instance, Bash scripts use `#!/bin/bash` and Python scripts use `#!/usr/bin/env python`.

9.2 Interaction with the Daemon

The My Daemon installation screen is shown in Figure 4 and Figure 5 for a CB3 robot and a e-Series robot, respectively. The code can be found in Listing 19, page 62, in Appendix B. [My Daemon Program and Installation Node](#).

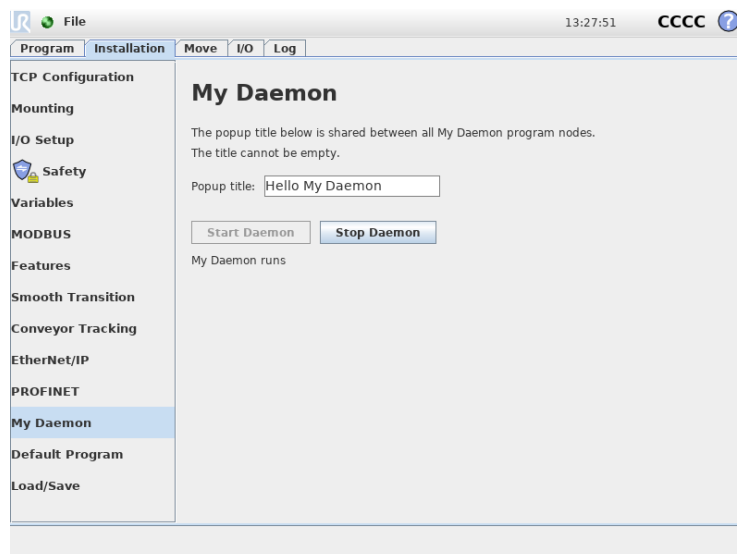


Figure 4: My Daemon installation screen on a CB3 robot

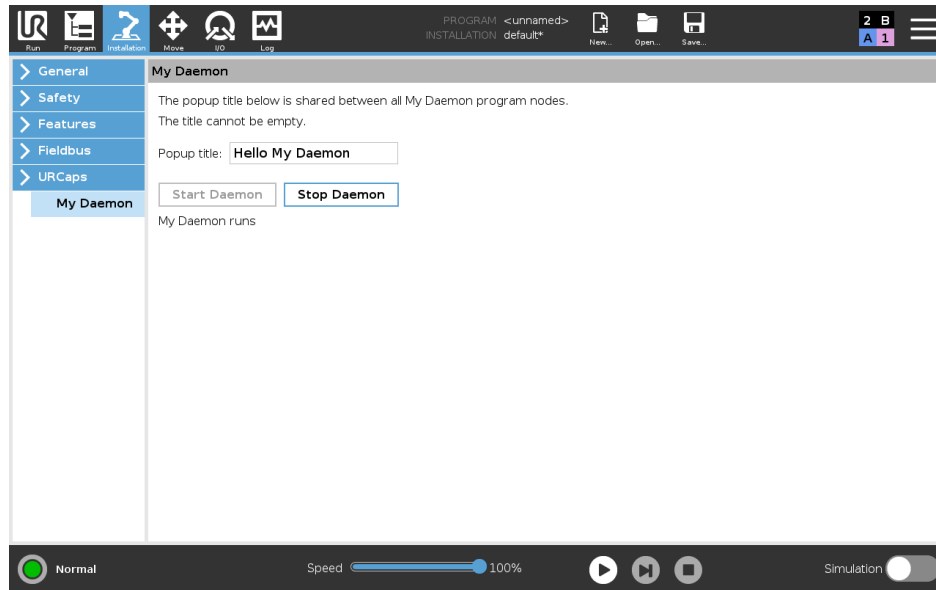


Figure 5: My Daemon installation screen on a e-Series robot

Two buttons have been added to the installation screen to enable and disable the daemon. In this example the daemon is enabled by default when a new installation is created, and future changes to the desired run state will be stored in the data model.

The daemon runs in parallel with PolyScope and can in principle change its state independently. Therefore, the label below the buttons displays the current run status of the daemon. This label is updated with a 1 Hz frequency, utilizing the `java.util.Timer` class. Since the UI update is initiated from a different thread than the Java AWT thread, the timer task must utilize the `EventQueue.invokeLater` functionality. Note, the `Timer` is added when the My Daemon Installation screen is opened (see `openView()`) and removed when the user moves away from the screen (see `closeView()`) to conserve computing resources.

Two options are available for Java and URScript to communicate with the daemon:

- TCP/IP sockets can be used to stream data.
- XML encoded Remote Procedure Calls (XML-RPC) can be used for configuration tasks (e.g. camera calibration) or service execution (e.g. locating the next object).

The advantage of XML-RPC over sockets is that no custom protocol or encoding needs to be implemented. The URScript XML-RPC implementation supports all URScript data types. Moreover, a RPC will only return when the function execution has been completed. This is desirable when the next program step relies on data retrieved from the daemon service. Plain sockets are on the other hand more efficient for data streaming, since there is no encoding overhead. Both methods can be complimentary applied and are available for Java ↔ daemon and URScript ↔ daemon communication.

Listing 10: URScript XML-RPC example

```
1 global mydaemon_swing = rpc_factory("xmlrpc", "http://127.0.0.1:40405/RPC2")
2 global mydaemon_message = mydaemon_swing.get_message("Bob")
3 popup(mydaemon_message, "My Title", False, False, blocking=True)
```

Listing 10 shows a small URScript example for making a XML-RPC call to a XML-RPC server.

The `hello-world.py` example daemon (see Listing 22, page 70) can be used as XML-RPC test server. Simply start the daemon in the My Daemon and run the URScript in a `Script` node.

The intention of this URScript example is to retrieve a message from the daemon to display during runtime (similar to the My Daemon program node). The `rpc_factory` script function creates a connection to the XML-RPC server in the daemon. The new connection is stored in the global `my_daemon_swing` variable and serves as a handle. The next line then requests the XML-RPC server in the daemon to execute the `get_message(...)` function with the string argument `"Bob"` and return the result. The return value of the RPC call is stored in the `mydaemon_message` variable for further processing in the `popup(...)` script function.

Note, making XML-RPC calls from URScript does not require any additional function stubs or pre-definitions of the remote function to be executed in URScript. Until the XML-RPC returns this URScript thread is automatically blocked (i.e. no `sync` nor `Wait` is needed). The standard XML-RPC protocol does not allow `void` return values and XML-RPC extensions enabling this are not always compatible.

The My Daemon Swing example also includes a Java XML-RPC client example, see the combination of the `MyDaemonProgramNodeContribution` and `XMLRPCMyDaemonInterface` classes (Listing 20, page 65 and listing 21, page 67 respectively). Note, the execution of the XML-RPC calls is not on the main Java AWT thread, but offloaded to a separate thread.

9.3 C/C++ Daemon Executables

The CB3.0/3.1 and CB5.0 control boxes all run a minimal Debian 32-bit Linux operating system. To guarantee binary compatibility all C/C++ executables should be compiled with the `urtool3` cross-compiler under Linux. The `urtool3` cross-compiler is included in the SDK installation.

To test if the `urtool3` is properly installed type the following in a terminal:

```
1 echo $URTOOL_ROOT; i686-unknown-linux-gnu-g++ --version
```

The correct output is:

```
1 /opt/urtool-3.0
2 i686-unknown-linux-gnu-g++ (GCC) 4.1.2
3 Copyright (C) 2006 Free Software Foundation, Inc.
4 This is free software; see the source for copying conditions. There is NO
5 warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

If the first line is not printed directly after installing the SDK, please reboot your PC for the environment variables to be updated.

The My Daemon Swing URcap comes with a fully functional C++ XML-RPC server example that is equivalent to the `hello-world.py` Python daemon. Simply switch the comments in the function `getExecutable()` in the `MyDaemonDaemonService` class (Listing 9, page 33), and recompile to use the C++ daemon implementation. The popup title should now be appended with `"(C++)"` instead of `"(Python)"` during execution of the URcap.

The C++ daemon directory structure is shown in Figure 6, page 37. For managing the software construction process of the C++ daemon a tool called Scons is used. The `SConstruct` file among other things contains the main configuration, the `urtool3` cross-compiler, and `libxmlrpc-c` integration. The `SConscript` files are used to define the compilation targets, e.g. the Hello World binary.

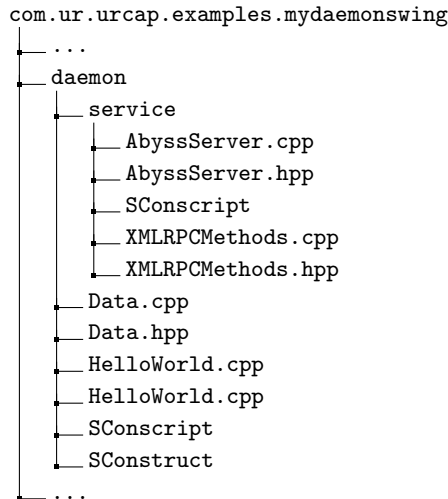


Figure 6: Structure of the C++ daemon of the My Daemon Swing project

For the example URCap, the daemon will be build as part of the URCap build process by maven. However, the daemon can also be compiled manually by typing the following in a terminal:

```

1 cd com.ur.urcap.examples.mydaemonswing/daemon
2 scons release=1

```

This will build a release version of the daemon. Using `release=0` will build an executable with debugging symbols.

The XML-RPC functionality in the C++ daemon relies on the open-source library `libxmlrpc-c` (<http://xmlrpc-c.sourceforge.net>). This library is by default available on the CB3.0/3.1 and CB5.0 control boxes. The `service` directory contains all relevant XML-RPC code. The `AbyssServer` is one of the XML-RPC server implementations supported by `libxmlrpc-c`. Please look in the C++ code for more programming hints and links to relevant documentation.

9.4 Tying the different Contributions together

The new My Daemon Swing URCap installation node, program node, and daemon executable are registered and offered to PolyScope through the code in Listing 11.

Three services are registered:

- `MyDaemonInstallationNodeService`
- `MyDaemonProgramNodeService`
- `MyDaemonDaemonService`

The `MyDaemonInstallationNodeService` class has visibility to an instance of the `MyDaemonDaemonService` class. This instance is passed in the constructor when a new installation node instance of the type `MyDaemonInstallationNodeContribution` is created with the `createInstallationNode(...)` method. In this way, the daemon executable can be controlled from the installation node.

Listing 11: Tying different URCap contributions together

```
1 package com.ur.urcap.examples.mydaemonswing.impl;
2
3 import com.ur.urcap.api.contribution.DaemonService;
4 import com.ur.urcap.api.contribution.installation.swing.
    SwingInstallationNodeService;
5 import com.ur.urcap.api.contribution.program.swing.SwingProgramNodeService;
6 import org.osgi.framework.BundleActivator;
7 import org.osgi.framework.BundleContext;
8
9 public class Activator implements BundleActivator {
10     @Override
11     public void start(final BundleContext context) throws Exception {
12         MyDaemonDaemonService daemonService = new MyDaemonDaemonService();
13         MyDaemonInstallationNodeService installationNodeService = new
            MyDaemonInstallationNodeService(daemonService);
14
15         context.registerService(SwingInstallationNodeService.class,
            installationNodeService, null);
16         context.registerService(SwingProgramNodeService.class, new
            MyDaemonProgramNodeService(), null);
17         context.registerService(DaemonService.class, daemonService, null);
18     }
19
20     @Override
21     public void stop(BundleContext context) throws Exception {
22     }
23 }
```

10 URCap Examples Overview

This section contains a short description of each of the URCap examples included with the URCaps SDK.

10.1 Regular Samples

Hello World Swing serves as the primary example throughout this tutorial and introduces all the core concepts of a URCap. This includes contributions to PolyScope of program nodes and installation nodes that seamlessly hook into:

- The UI
- Persistence of program and installation files
- Creation and execution of programs
- Program undo/redo functionality

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* SwingProgramNodeService, ProgramNodeContribution, SwingProgramNodeView, ContributionProvider, SwingInstallationNodeService, InstallationNodeContribution, SwingInstallationNodeView, UndoRedoManager, DataModel, ScriptWriter.

My Daemon Swing is an extension to the Hello World Swing URCap and demonstrates how a Python 2.5 or C++ daemon can be integrated with the URCap Software Platform. This is useful when a URCap depends on e.g. a driver or server which is not implemented in Java. Furthermore, the URCap shows how the XML-RPC protocol can be used to communicate with the daemon from an installation node and in the script code generated by a program node.

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* DaemonContribution, DaemonService.

Script Function Swing demonstrates how to add functions to the list of available script functions in the Expression Editor. Script functions often used by end users of a URCap should be added to this list.

Information:

- *Available from:*
 - URCap API version 1.3.0.

- PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* Function, FunctionModel

Pick or Place Swing is a toy example that shows how to make changes to the program tree through the `TreeNode` API. The program node service (`SwingProgramNodeService` interface) is configured in such a way that it creates program node contributions that can only be inserted into the program tree by a URCap and not from the UI of PolyScope by the end user.

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* ProgramModel, TreeNode, ProgramNodeFactory, SwingProgramNodeService, ContributionConfiguration

Ellipse Swing is a toy example, where a pose is used to define the center point for an ellipse-like movement. The movement is achieved by inserting a pre-configured `MoveP` program node containing pre-defined and named Waypoint nodes. This example demonstrates how to:

- Obtain a pose for the robot position by requesting the end user to define it using the Move Tab
- Name waypoints
- Request the end user to move the robot to a given target position
- Allow the end user to use the builtin PolyScope support for starting from and pausing/breaking on a selected program node in the program tree. In this case, the end user can start from or break on a specific Waypoint child node under the Ellipse (URCap) program node.

Note:

- The functionality of assigning the Waypoint nodes custom names is only available from URCap API version 1.4.0 (released with PolyScope version 3.7.0/5.1.0)
- Requesting the user to move the robot to a defined center point is only available from URCap API version 1.5.0 (released with PolyScope version 3.8.0/5.2.0).
- From URCap API version 1.6.0 (released with PolyScope version 3.9.0/5.3.0) the use of the deprecated move node config factory (`MoveNodeConfigFactory` interface) has been replaced with the equivalent builder and the TCP selection of the `MoveP` node is pre-configured.
- Support for allowing the end user to start from and break on child nodes is only available from URCap API version 1.9.0 (released with PolyScope version 5.6.0).
- From URCap API version 1.11.0 (released with PolyScope 3.14.0/5.9.0) the use of the deprecated method `getUserDefinedRobotPosition(RobotPositionCallback)` has been replaced with the equivalent `getUserDefinedRobotPosition(RobotPositionCallback2)` method. Furthermore, the use of the deprecated factory method for creating fixed position Waypoint node configurations (`createFixedPositionConfig(...)` without TCP offset) has been replaced with the equivalent method taking the TCP offset as parameter as well.

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* UserInteraction, RobotPositionCallback2, RobotMovement, RobotMovementCallback, WaypointNodeConfig, MovePMoveNodeConfig, MoveNodeConfigBuilders, MovePConfigBuilder, PoseFactory, Pose, SimpleValueFactory, JointPositions

Cycle Counter Swing demonstrates how to work with variables. In this example, the chosen variable will be incremented each time the program node is executed.

This sample also demonstrates how to allow the end user to use the builtin PolyScope support for starting from and pausing/breaking on a selected program node in the program tree. In this case, the end user can start from or break on any child node under the Cycle Counter (URCap) program node.

Note:

- Support for allowing the end user to start from and break on child nodes is only available from URCap API version 1.9.0 (released with PolyScope version 5.6.0)

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* Variable, VariableFactory, ExpressionBuilder

Idle Time Swing demonstrates how to work with the ProgramNodeVisitor to traverse all program nodes in a sub-tree. In this example, all Wait nodes will be visited. If a Wait node is configured to wait for an amount of time, that amount of idle time (in seconds) will accumulate in the selected variable.

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* ProgramNodeVisitor, WaitNodeConfig

Localization Swing demonstrates how to implement localization in URCaps. PolyScope localization settings is accessed through the SystemSettings API.

Information:

- *Available from:*
 - URCap API version 1.3.0.

- PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* SystemSettings, Localization, Unit, SimpleValueFactory

User Input demonstrates how to work with the virtual on-screen keyboard/keypad and user input validation.

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 3.6.0/5.0.4.
- *Main API interfaces:* KeyboardInput, InputValidationFactory

My Toolbar demonstrates how to implement a PolyScope toolbar contribution.

Information:

- *Available from:*
 - URCap API version 1.3.0.
 - PolyScope version 5.0.4.
- *Main API interfaces:* SwingToolbarService, SwingToolbarContribution.

Node Ordering Swing demonstrates how to define a specific sort order in PolyScope for the program node contributions from a URCap.

Information:

- *Available from:*
 - URCap API version 1.5.0.
 - PolyScope version 3.8.0/5.2.0.
- *Main API interfaces:* ContributionConfiguration, SwingProgramNodeService.

Tool Changer Swing is a toy example that shows how contribute TCPs to PolyScope as well as access the list of available TCPs in PolyScope.

In the installation node contribution, the user can define a tool change position, enable/disable different tool TCPs and define a translational offset between the tool flange and all the enabled tool TCPs.

In the program node contribution, the user can select a TCP for the new tool from the list of all available TCPs in PolyScope. When the program node is executed, the robot will move to the user-defined tool change position, change the tool (simulated by a waiting period) and finally change the active TCP to the selected TCP.

Information:

- *Available from:*
 - URCap API version 1.5.0.

- PolyScope version 3.8.0/5.2.0.
- *Main API interfaces:* TCPContributionModel, TCPModel.

Note:

- From URCap API version 1.11.0 (released with PolyScope 3.14.0/5.9.0) the use of the deprecated method `getUserDefinedRobotPosition(RobotPositionCallback)` has been replaced with the equivalent `getUserDefinedRobotPosition(RobotPositionCallback2)` method.

Move Until Detection Swing demonstrates how to work with the Distance and Until program nodes through the URCap API.

The program node contribution creates a Direction node that moves the robot downwards, until a sensor is triggered (through an input), or until a maximum distance is reached. If the sensor is triggered, a connected physical device (e.g. a gripper) is activated by setting an output high. If the maximum distance is travelled before the sensor is triggered, a popup is generated to display an error to the user. The user can specify the maximum distance through a text field in the Move Until Detection node.

Information:

- *Available from:*
 - URCap API version 1.6.0.
 - PolyScope version 3.9.0/5.3.0.
- *Main API interfaces:* DirectionNode, DirectionNodeConfigBuilder, UntilNode, UntilNodeConfigFactory.

Tool I/O Control Swing demonstrates how to work with the resource model and request exclusive control of the Tool I/O Interface through the URCap API. It also demonstrates how to check, if a specific capability is available on the underlying robot system (in this case the Tool Communication Interface (TCI) and Tool Output Mode features).

For further details about resource control, please see the separate [Resource Control](#) document. The concept of system capabilities and checking their availability is described in the separate [Capabilities](#) document.

Information:

- *Available from:*
 - URCap API version 1.7.0.
 - PolyScope version 3.10.0/5.4.0.
- *Main API interfaces:* ResourceModel, ControllableResourceModel, ToolIOInterfaceController, ToolIOInterface, ToolIOInterfaceControllable, CapabilityManager.

Grip and Release Swing demonstrates how to use gripper devices in a template program node.

In the program node contribution, the user can select a gripper among the registered grippers available in PolyScope. When a gripper has been selected, the selected gripper can be applied to the template node. This will insert two Gripper program nodes for

the selected gripper: one node configured for a grip action and one node configured for a release action.

This example demonstrates how to:

- Get the list of grippers available in PolyScope
- Insert a Gripper program node for a specific gripper device in the program tree
- Configure a Gripper program node for grip and release actions

Note: To have grippers to select from in the program node, some of the gripper driver URCap samples (e.g. Simple Gripper) can be installed.

For further details about how to use devices in template nodes, please see the separate [Using a Device in a Template](#) document.

Information:

- *Available from:*
 - URCap API version 1.9.0.
 - PolyScope version 3.12.0/5.6.0.
- *Main API interfaces:* GripperManager, GripperProgramNodeFactory, GripperNode, GripConfigBuilder, ReleaseConfigBuilder, GripperDevice.

Create Feature Swing is an example that shows how to contribute a feature to PolyScope as well as how to modify and remove it.

In the installation contribution, the user can define a feature and subsequently update and remove it. The created feature is stored in the data model of the installation contribution.

In the program node contribution, the user will be asked to create a feature from the installation contribution, if not done already. When the feature is created, the user can press a button to create a movement relative to the created feature. The movement is achieved by inserting a pre-configured MoveL program node containing pre-defined Waypoint nodes. When the program node is executed, the robot will perform a square movement centered at the created feature.

Information:

- *Available from:*
 - URCap API version 1.9.0.
 - PolyScope version 3.12.0/5.6.0.
- *Main API interfaces:* FeatureContributionModel.

Note:

- From URCap API version 1.11.0 (released with PolyScope 3.14.0/5.9.0) the use of the deprecated method `getUserDefinedRobotPosition(RobotPositionCallback)` has been replaced with the equivalent `getUserDefinedRobotPosition(RobotPositionCallback2)` method. Furthermore, the use of the deprecated factory method for creating fixed position Waypoint node configurations (`createFixedPositionConfig(...)` without TCP offset) has been replaced with the equivalent method taking the TCP offset as parameter as well. The tool flange is used as the TCP offset.

Create Payload Swing is an example that shows how to contribute a payload to PolyScope as well as how to modify and remove it.

In the installation contribution, the user can create a payload and subsequently update the mass and center of gravity (CoG) parameters of the created payload as well as remove the payload. Values entered by the user for the mass and center of gravity (CoG) parameters are validated using the parameter limits available through the API. The created payload is stored in the data model of the installation contribution.

In the program node contribution, the user will be asked to create a payload from the installation contribution, if not done already. When the payload is created, the user can press a button to create a Set Payload node with the payload (created in the installation contribution) pre-selected.

Information:

- *Available from:*
 - URCap API version 1.13.0
 - PolyScope version 5.11.0
- *Main API interfaces:* PayloadContributionModel, RobotLimits, SetPayloadNode

10.2 Driver Samples

Custom User Inputs demonstrates how to use different types of user inputs (e.g. combo box and checkbox inputs) and other (non-user input) UI elements (e.g text components) when defining a custom UI for a driver contribution.

The URCap also shows how to use the filler UI element for controlling/grouping the layout and how to add a custom input validator for detecting errors for enterable user inputs, in this case an IP address user input (text field).

Information:

- *Available from:*
 - URCap API version 1.7.0.
 - PolyScope version 3.11.0/5.5.0.
- *Main API interfaces:* CustomUserInputConfiguration.

Simple Gripper demonstrates how to create a gripper driver contribution for a basic gripper that only supports the mandatory "default" grip and release actions. The URCap uses digital outputs in the script code generation to trigger grip and release actions.

For further details about gripper driver contributions, please see the separate [Gripper Driver](#) document.

Information:

- *Available from:*
 - URCap API version 1.8.0.
 - PolyScope version 3.11.0/5.5.0.

- *Main API interfaces:* GripperContribution.

Advanced Gripper demonstrates how to create a gripper driver contribution for a more advanced gripper that supports some of the optional gripper capabilities and controls the Output Voltage setting of the Tool I/O Interface resource. The URCap shows how to:

- Configure gripper capabilities for, e.g. width, speed, force, vacuum and feedback for grip and release detection
- Request exclusive control of the Tool I/O Interface resource
- Configure the Output Voltage I/O setting of the Tool I/O Interface

Additional information:

- For details about gripper driver contributions, please see the separate [Gripper Driver](#) document.
- For information about system resource control, please see the separate [Resource Control](#) document.
- To see an example of how some of the other settings of the Tool I/O Interface can be configured, see the [Tool I/O Control Swing](#) URCap example.

Note:

- Feedback capabilities of the gripper is only available from URCap API version 1.9.0 (released with PolyScope version 3.12.0/5.6.0)

Information:

- *Available from:*
 - URCap API version 1.8.0.
 - PolyScope version 3.11.0/5.5.0.
- *Main API interfaces:* GripperContribution, GripperCapabilities, SystemConfiguration, GripActionParameters, ReleaseActionParameters, GripperFeedbackCapabilities.

Custom Gripper Setup demonstrates how to create a gripper driver contribution which defines a custom UI in the installation node for setting up the gripper as well as adds a TCP for the gripper to PolyScope.

For further details about gripper driver contributions, please see the separate [Gripper Driver](#) document.

Information:

- *Available from:*
 - URCap API version 1.8.0.
 - PolyScope version 3.11.0/5.5.0.
- *Main API interfaces:* GripperContribution, TCPConfiguration, CustomUserInputConfiguration.

Dual Zone Gripper demonstrates how to create a gripper driver contribution for a multi-gripper that supports multiple permanently enabled, individual grippers as well as dynamically adjusts the parameters of a capability (after it has been registered) for all individual grippers.

In this example, the gripper is a vacuum dual gripper with two independent physical grippers (zones) "built into" the gripping device. The URCap provides the user the option to select from three individual zones (grippers), named *Zone A*, *Zone B* and *Zone A+B*. The user can operate *Zone A* and *Zone B* independently of one another or choose to use both zones at the same by selecting *Zone A+B*.

A custom UI is defined in the installation node which allows the user to enable the "Use Fragile Handling" option. When enabled, this will restrict the maximum vacuum level available for a grip action. This is achieved by reducing the maximum value and default value parameters of the registered vacuum capability simultaneously for all zones.

The URCap shows how to:

- Configure the multi-gripper capability to support multiple permanently enabled, individual grippers
- Add a dedicated TCP for each individual gripper (zone)
- Dynamically update the parameter values of a registered (parameter-based) capability for all individual grippers (zones). In this case, the registered capability is the grip vacuum capability.

For further details about gripper driver contributions, please see the separate [Gripper Driver](#) document.

Information:

- *Available from:*
 - URCap API version 1.11.0.
 - PolyScope version 3.14.0/5.9.0.
- *Main API interfaces:* GripperContribution, GripperCapabilities, GripperListProvider, GripperListBuilder, SelectableGripper, SystemConfiguration, TCPConfiguration, GripVacuumCapability

Dynamic Multi-Gripper demonstrates how to create a gripper driver contribution that supports both a single gripper setup and multi-gripper setup as well as dynamically adjusts the parameters of a capability (after it has been registered) exclusively for each individual gripper.

In this example, the gripper driver provides the user the option of choosing between using a setup with only a single gripper mounted on the robot, or one where two separate, identical grippers are mounted on the robot at the same time. Each individual gripper supports moving to a user configurable position (open/close to a configurable width).

A custom UI is defined in the installation node which allows the user to select how many gripper are mounted on the robot. The available options are *Single* and *Dual*. Selecting the *Single* option will disable the secondary gripper, *Gripper 2*, and only the standard

gripper, *Gripper 1*, will be available to the user. Choosing the *Dual* option will enable both grippers making both of them available to the user. Depending on the option selected, the offset of TCP for the standard gripper (*Gripper 1*) is updated accordingly.

For each individual gripper (*Gripper 1* and *Gripper 2*), the user can configure the type of fingertips attached to the gripper (in the installation node). There are two options, *Standard* and *Extended*. The *Standard* option is the default whereas the *Extended* option can be selected, when fingertips with wide range is attached. Selecting the *Extended* option will increase the maximum value and default value parameters of the registered width capability exclusively for the specific individual gripper (independently of the other gripper).

The URCap shows how to:

- Configure the multi-gripper capability to support multiple individual grippers where some of the grippers are initially disabled
- Dynamically enable and disable individual grippers
- Add a dedicated TCP for each individual gripper
- Dynamically update the parameter values of a registered (parameter-based) capability exclusively for an individual gripper (independently of the other grippers). In this case, the registered capability is the width capability.

For further details about gripper driver contributions, please see the separate [Gripper Driver](#) document.

Information:

- *Available from:*
 - URCap API version 1.11.0.
 - PolyScope version 3.14.0/5.9.0.
- *Main API interfaces:* GripperContribution, GripperCapabilities, GripperListProvider, GripperListBuilder, SelectableGripper, MultiGripperCapability, SystemConfiguration, TCPConfiguration, WidthCapability

Simple Screwdriver demonstrates how to create a screwdriver driver contribution for a basic screwdriver that only supports the mandatory start and stop screwdriver operations. The URCap uses digital outputs in the script code generation to start and stop the screwdriver.

For further details about screwdriver driver contributions, please see the separate [Screwdriver Driver](#) document.

Information:

- *Available from:*
 - URCap API version 1.9.0.
 - PolyScope version 5.6.0.

Advanced Screwdriver demonstrates how to create a screwdriver driver contribution that supports some of the optional operation and operation feedback capabilities, e.g. Program Selection, Feed Screw, Drive Screw OK and Screwdrive Ready. The URCap primarily uses

I/Os for the implementation of the capabilities.

For further details about screwdriver driver contributions, please see the separate [Screwdriver Driver](#) document.

Information:

- *Available from:*
 - URCap API version 1.9.0.
 - PolyScope version 5.6.0.

Custom Screwdriver demonstrates how to create a screwdriver contribution driver which defines a custom UI in the Screwdriving installation node for setting up the screwdriver as well as adds a default TCP for the screwdriver to PolyScope.

For further details about screwdriver driver contributions, please see the separate [Screwdriver Driver](#) document.

Information:

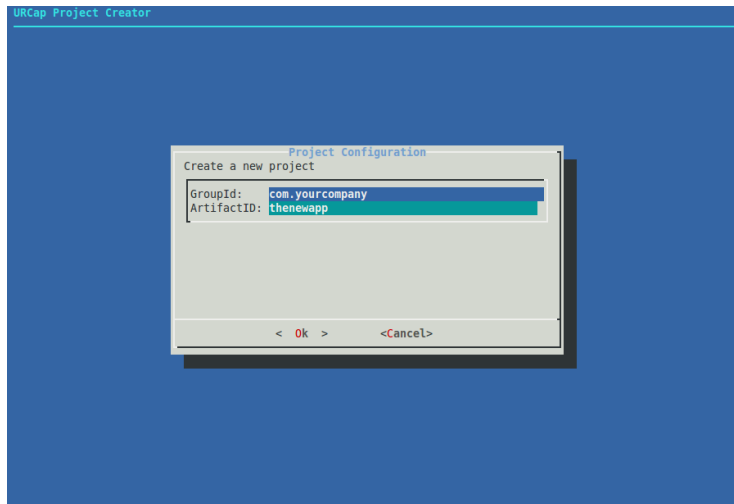
- *Available from:*
 - URCap API version 1.9.0.
 - PolyScope version 5.6.0.

11 Creating new thin Projects using a Maven Archetype

There are different ways to get started with URCap development. One is to start with an existing URCap project and modify that. When you have got a hang of it you may want to start with an empty skeleton with the basic Maven structure. So enter the directory of the URCaps SDK and type:

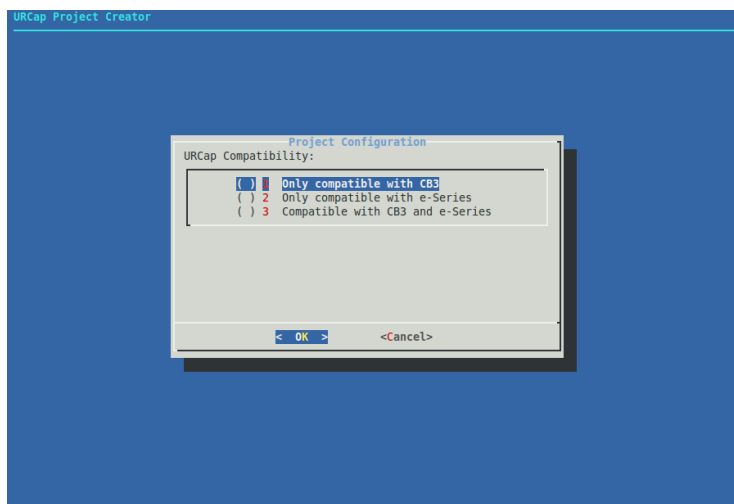
```
1 $ ./newURCap.sh
```

This prompts you with a dialog box where you select a group and artifact-id for your new URCap:



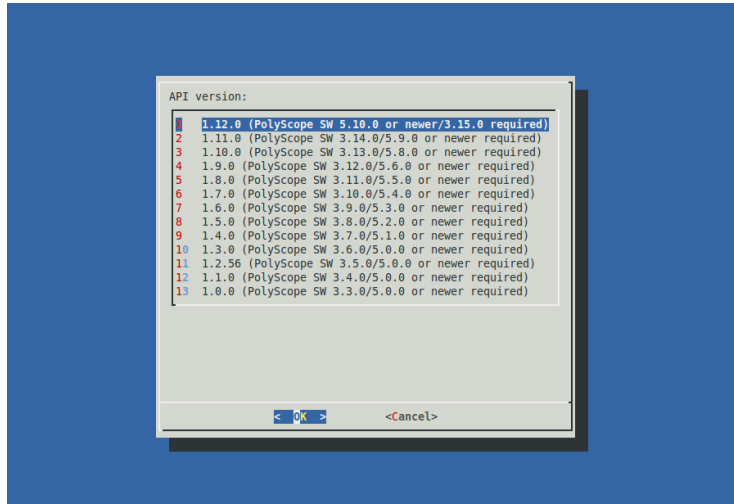
An example could be `com.yourcompany` as group-id and `thenewapp` as artifact-id. Consult best practices naming conventions for Java group-ids.

Next you must specify which Universal Robots robot series your URCap is compatible with:



Maybe your URCap requires specific hardware and/or software features, which are only available on a specific robot series (e.g., e-Series robots only), or the URCap might be compatible with all robot series. Such considerations should determine your choice here. See section [12.1. Robot Series Compatibility](#) for more details about robot series compatibility.

Finally, you must also specify the target URCap API version. Choosing an earlier version of the API will make your URCap compatible with earlier PolyScope versions, but also limit the functionality accessible through the API. URCap API compatibility is described in section [12.2. API Compatibility](#).



Pressing the Ok button in the last dialog creates a new Maven project under the sub-folder `./com.yourcompany.thenewapp`. This project can easily be imported into an IDE for Java, e.g. Eclipse, Netbeans, or IntelliJ.

Notice that the generated `pom.xml` file has a section with a set of properties for the new URCap with meta-data for vendor, contact address, copyright, description, and short license information which will be displayed to the user when the URCap is installed in PolyScope. This section also contains boolean compatibility flag properties specifying the URCap's compatibility with the different available robot series. Update the URCap meta-data section with the data relevant for the new URCap. See Listing 1 for an example of how this section might look.

Should you need to change the version of the URCap API to depend upon after your project has been setup, this can be done in the `pom.xml` file in your project. Here you must update to the desired version in the URCap API dependency under the `<dependencies>` section of the `pom.xml`-file as well as modify the `<import-package>` element under the `maven-bundle-plugin` if version information is present. See listings 12, 13 and 16 for examples of this

Listing 12: Modify URCap API runtime dependency in `pom.xml`

```

1      ...
2      <Import-Package>
3          com.ur.urcap.api*,
4          *
5      </Import-Package>
6      ...

```

Listing 13: Specifying URCap API compile time dependency in pom.xml

```
1    ...
2    <dependencies>
3    ...
4    <dependency>
5        <groupId>com.ur.urcap</groupId>
6        <artifactId>api</artifactId>
7        <version>1.0.0.30</version>
8        <scope>provided</scope>
9    </dependency>
10   ...
11 </dependencies>
12 ...
```

12 Compatibility

12.1 Robot Series Compatibility

When developing URCaps you must mandatorily specify which robot series your URCap is compatible with, i.e. which robot series your URCap can run on. The purpose is to provide a good user experience for the URCap installation process by ensuring that end users cannot install a URCap that is not compatible with their robot and/or Universal Robots offline simulator software (URSim).

When specifying your URCap's compatibility, you need to consider, if your URCap depends on specific hardware and/or software functionality, which is not available on all robot series (e.g., only available on e-Series robots), or if the URCap can run on all available robot series. Note that your URCap's compatibility must be specified for all robot series available in the line of robots from Universal Robots (including robot series which your URCap is incompatible with).

12.1.1 Compatibility Flags

The URCap's compatibility is specified using boolean compatibility flags which are defined as properties in the `pom.xml` Maven configuration file in your URCap project. Two compatibility flag properties, `urcap.compatibility.CB3` and `urcap.compatibility.eSeries`, must be present in the URCap meta-data section in the `pom.xml` file. An example of the meta-data section with the two compatibility flag properties is shown in Listing 14.

Listing 14: Compatibility flags in the `pom.xml` file's URCap meta-data properties section

```

1  <!--*****-->
2  <!--      Note: Update this section with relevant meta data      -->
3  <!--      that comes along with your URCap                      -->
4  <!--*****-->
5  <!--***** BEGINNING OF URCAP META DATA *****-->
6  <urcap.symbolicname>com.ur.urcap.examples.helloworldswing</urcap.symbolicname>
7  <urcap.vendor>Universal Robots</urcap.vendor>
8  <urcap.contactAddress>Energivej 25, 5260 Odense S, Denmark</urcap.
   contactAddress>
9  <urcap.copyright>Copyright (C) 2009-2025 Universal Robots. All Rights Reserved
   </urcap.copyright>
10 <urcap.description>Hello World Swing sample URCap</urcap.description>
11 <urcap.licenseType>Sample license</urcap.licenseType>
12 <urcap.compatibility.CB3>true</urcap.compatibility.CB3>
13 <urcap.compatibility.eSeries>true</urcap.compatibility.eSeries>
14 <!--***** END OF URCAP META DATA *****-->

```

These two properties specify the URCap's compatibility with the CB3 and the e-Series robot series, respectively. Acceptable values for the properties are either `true` or `false`. A value of `true` indicates that the URCap can be installed and run on the given robot series, and a value of `false` indicates otherwise. Note that both flags must present including the flag for a robot series which your URCap is not compatible with.

Using the `newURCap.sh` script mentioned in section 11. [Creating new thin Projects using a Maven Archetype](#), the compatibility flags are handled automatically for you. On the other hand, if an existing URCap needs to be updated to support the compatibility flags, you must manually update your URCap project's `pom.xml` file. This will only be necessary, if you used a version of the URCap SDK lower than 1.12.0 to create the URCap by either using the `newURCap.sh` script, or by basing the URCap on a modification of one of the URCap example projects included in the SDK. The required changes are described in subsection 12.1.3. [Updating Existing URCaps](#).

If the compatibility of your URCap changes, remember to update the value of the URCap's compatibility flags accordingly.

12.1.2 Installation and Startup of URCaps

When the end user attempts to install a new URCap through the URCap Setup/Settings screen in PolyScope, PolyScope will inspect the URCap's compatibility flags. Depending on the boolean value of the URCap's compatibility flags and the type of series of the end user's robot (or offline simulator), the user will either be allowed to install or prevented from installing that URCap. If the URCap is incompatible, the end user will be informed of this through a dialog.

Similarly, a URCap's compatibility flags are examined during startup of PolyScope, if the URCap was deployed and installed using Maven (described in section 6. [Deployment with Maven](#)) instead of being installed through the PolyScope UI. If the URCap is incompatible, it will not be allowed to start and an error dialog will be displayed in PolyScope.

The presence of the compatibility flags in a URCap will be verified during startup of PolyScope (if the URCap was deployed and installed using Maven), and a URCap will not be started, if any of the flags are missing. Likewise, a URCap with missing compatibility flag(s) will not be installable through PolyScope. An error dialog will be displayed in PolyScope in these cases.

Note that only URCaps depending on URCap API version 1.12.0 or higher will be prevented from starting or being installed, if the compatibility flags are not present.

12.1.3 Updating Existing URCaps

If you have an existing URCap that needs to be updated to support the compatibility flags, changes must be applied to your URCap project's `pom.xml` file.

First of all, you must add the two compatibility flag properties, `urcap.compatibility.CB3` and `urcap.compatibility.eSeries`, to the URCap meta-data section as shown in Listing 14 in section 12.1.1. [Compatibility Flags](#). Remember to specify boolean values corresponding to your URCap's compatibility.

Listing 15: Excerpt of the `pom.xml` file's Maven bundle plugin configuration with compatibility flag support

```

1    ...
2    <build>
3      <plugins>
4        ...
5        <plugin>
6          <groupId>org.apache.felix</groupId>
7          <artifactId>maven-bundle-plugin</artifactId>
8          ...
9          <configuration>
10           <instructions>
11             <!--***** DO NOT MODIFY THE ENTRIES OF THIS SECTION *****-->
12             <Bundle-Category>URCap</Bundle-Category>
13             ...
14             <URCapCompatibility-CB3>${urcap.compatibility.CB3}</
              URCapCompatibility-CB3>
15             <URCapCompatibility-eSeries>${urcap.compatibility.eSeries}</
              URCapCompatibility-eSeries>
16             <!--***** DO NOT MODIFY THE ENTRIES OF THIS SECTION *****-->
17             ...
18             <instructions>
19             <configuration>
20             <plugin>

```

```

21     ...
22     <plugins>
23     ...

```

In order for PolyScope to be able to read the specified values for the compatibility flag properties, the `<build>` section with configuration of build plugins also needs to be updated. Here the configuration for the Maven bundle plugin must be updated to contain two new instructions, `<URCapCompatibility-CB3>` and `<URCapCompatibility-eSeries>`, as shown in the excerpt in Listing 15.

For convenience, instead of manually making the changes described above, you can use the `upgradeURCap.sh` script to update your `pom.xml` file.

Below are examples of usage of the script, where `/home/myuser/myURCap` is the path to the folder with a URCap project and `/home/myuser/sdk` is the path to the folder of the URCap SDK.

Go to the folder of the URCap SDK and type:

```
1 $ ./upgradeURCap.sh /home/myuser/myURCap
```

This will apply the required changes to the URCap project's `pom.xml` file inside the folder `/home/myuser/myURCap`. Both compatibility flag properties are assigned the boolean value of `true` by default (i.e. your URCap is compatible with both the CB3 and e-Series robot series). Remember to change the value of the compatibility flag properties in the `pom.xml` file accordingly, if the preassigned default values does not match your URCap's compatibility.

Instead of specifying an absolute path to your URCap project folder, you can also specify a relative path:

```
1 $ ./upgradeURCap.sh ../myURCap
```

Alternatively, it is also possible to copy the `upgradeURCap.sh` script to your URCap project folder and execute the script from within that folder by typing:

```

1 $ cd myURCap
2 $ ./upgradeURCap.sh .

```

Finally, you have the option of directly specifying the boolean value for the compatibility flags using the `compatibilityCB3` and `compatibilityESeries` arguments:

```
1 $ ./upgradeURCap.sh ../myURCap compatibilityCB3=false compatibilityESeries=true
```

In this case, you have specified that your URCap is compatible with the e-Series robot series, but incompatible with the CB3 robot series.

12.2 API Compatibility

When developing URCaps you must specify a dependency on a version of the URCap API to compile against. Using the `newURCap.sh` script mentioned in section 11. [Creating new thin Projects using a Maven Archetype](#), this is handled automatically for you. How to update the version of the URCap API dependency for an existing URCap project is described at the end of section 11. [Creating new thin Projects using a Maven Archetype](#).

A given version of the API is compatible with a specific version of PolyScope (see table 1 below). PolyScope will remain backwards compatible with earlier versions of the API. This means that if you choose to use the newest API, customers using your URCap must be running at least the

version of PolyScope with which the given API was released.

It is not a problem if the customer is running a newer (future) version of PolyScope. However, it is not possible for the customer to use your URCap if he is running an earlier version of PolyScope than the one the API was released with. A good rule of thumb is thus to choose the earliest possible version of the API that supports the functionality you wish to use. This will target the broadest audience.

For instance, if you specify a dependency on the API version 1.1.0, your URCap will only run on PolyScope version 3.4.0 or newer. If you wish to target the broadest possible audience, you must use version 1.0.0 of the API and the customers must be running PolyScope version 3.3.0 or newer.

Note that API version 1.12.0 is the final API version supported on CB3 robots. Higher API versions are only available on e-Series robots.

URCap API version	Min. PolyScope version
1.18.0	5.23.0
1.17.0	5.21.0
1.16.0	5.20.0
1.15.0	5.18.0
1.14.0	5.15.0
1.13.0	5.11.0
1.12.0	3.15.0/5.10.0
1.11.0	3.14.0/5.9.0
1.10.0	3.13.0/5.8.0
1.9.0	3.12.0/5.6.0
1.8.0	3.11.0/5.5.0
1.7.0	3.10.0/5.4.0
1.6.0	3.9.0/5.3.0
1.5.0	3.8.0/5.2.0
1.4.0	3.7.0/5.1.0
1.3.0	3.6.0/5.0.4
1.2.56	3.5.0
1.1.0	3.4.0
1.0.0	3.3.0

Table 1: API versions and PolyScope version requirements

12.2.1 Advanced compatibility

Contrary to what is described above, it is possible to load a URCap depending on a newer API than what is officially supported by PolyScope. By configuring your URCap to resolve its dependencies runtime rather than install time, PolyScope will start your URCap regardless of the URCap API version dependency specified. Care must be taken to have a code execution path that does not use anything not available in the API that the given version of PolyScope supports (otherwise a `NoSuchMethodError` or `NoClassDefFoundError` will be thrown).

As an example you could have a dependency on API version 1.5.0 and run it on PolyScope version 3.7.0/5.1.0 (officially only supporting API version 1.4.0), but in the actual execution path you can only use types present in API version 1.4.0.

To have the URCap resolve at runtime the `pom.xml` must have the option `;resolution:=optional` appended to the URCap API entry in the `<import-package>` section. The full `<import-package>` section could look like this:

Listing 16: Excerpt of pom.xml file for advanced compatibility

```

1  ...
2  <Import-Package>
3      com.ur.urcap.api*;resolution:=optional,
4      *
5  </Import-Package>
6  ...

```

This will make the URCap start up regardless of what the actual dependency states.

As mentioned you must also structure your code so no code referring unsupported API functionality is executed. Also no `import` or `catch` clauses referring to unsupported types can be present in classes that will execute. See listings 17 and 18 for an example of how to structure this. In listing 17 all code related to unsupported API functionality is located and in 18 a check for PolyScope version number is performed before creating an instance of the `AdvancedFeature` class.

Listing 17: `AdvancedFeature` class showing advanced compatibility

```

1  import com.ur.urcap.api.contribution.installation.InstallationAPIProvider;
2  import com.ur.urcap.api.domain.InstallationAPI;
3  import com.ur.urcap.api.domain.value.Pose;
4  import com.ur.urcap.api.domain.value.PoseFactory;
5  import com.ur.urcap.api.domain.value.simple.Angle;
6  import com.ur.urcap.api.domain.value.simple.Length;
7
8  public class AdvancedFeature {
9      private final InstallationAPIProvider apiProvider;
10
11     public AdvancedFeature(InstallationAPIProvider apiProvider) {
12         this.apiProvider = apiProvider;
13     }
14
15     public void addTCP() {
16         InstallationAPI installationAPI = apiProvider.getInstallationAPI();
17         PoseFactory poseFactory = installationAPI.getValueFactoryProvider().
18             getPoseFactory();
19         Pose pose = poseFactory.createPose(0, 0, 100, 0, 0, 0, Length.Unit.MM,
20             Angle.Unit.RAD);
21         apiProvider.getInstallationAPI().getTCPContributionModel().addTCP("
22             GRIPPER_TCP", "Gripper", pose);
23     }
24 }

```

Listing 18: Usage of `AdvancedFeature` class

```

1      SoftwareVersion softwareVersion = apiProvider.getSystemAPI().
2          getSoftwareVersion();
3      if ((softwareVersion.getMajorVersion() == 3 && softwareVersion.
4          getMinorVersion() >= 8) ||
5          softwareVersion.getMajorVersion() == 5 && softwareVersion.
6          getMinorVersion() >= 2) {
7          new AdvancedFeature(apiProvider).addTCP();
8      }

```

If you choose to use this advanced feature, you must test your URCap carefully on all PolyScope versions you wish to support making sure all code execution paths are tested.

13 Exception Handling

All exceptions thrown and not caught in a URCap will be caught by PolyScope. If this happens when the end user selects either an installation node or a program node from the URCap, the UI provided by the URCap will be replaced by a screen displaying information about the error. In all other cases, a dialog will be shown to the end user.

The error screen and dialog will show that an exception has happened in the URCap along with meta information about the URCap. This occurs if the call stack originates from PolyScope (i.e. if the exception occurred when PolyScope called a method defined in a URCap API interface implemented by the URCap) or happens inside PolyScope (e.g. due to illegal arguments passed in an API method call). In this case, it will also contain a section showing the stack trace from the exception.

If an uncaught exception (e.g. a `NullPointerException`) happens in the source code of a URCap, PolyScope will try to identify the failing URCap. If successful, the error dialog will be shown. If PolyScope fails to identify the failing URCap, a general error dialog will be shown without meta information.

14 Troubleshooting

Internally in PolyScope, a URCap is installed as an OSGi bundle with the Apache Felix OSGi framework. For the purpose of debugging problems, it is possible to inspect various information about bundles using the Apache Felix command shell.

You can establish a shell connection to the running Apache Felix by opening a TCP connection on port 6666. Access the Apache Felix shell console by typing:

```
1  $ nc 127.0.0.1 6666
2
3  Felix Remote Shell Console:
4  =====
5
6  ->
```

Note that you need to use the `nc` command, since the `telnet` command is not available on the robot, and 127.0.0.1 because `localhost` does not work on a robot.

To view a list of installed bundles and their state type the following command:

```
1  -> ps
2  START LEVEL 1
3      ID      State      Level  Name
4  [ 0] [Active] [ 0] System Bundle (5.2.0)
5  [ 1] [Active] [ 1] aopalliance (1.0)
6  [ 2] [Active] [ 1] org.aspectj.aspectjrt (1.8.2)
7  [ 3] [Active] [ 1] org.netbeans.awtextra (1.0)
8  [ 4] [Active] [ 1] net.java.balloontip (1.2.4)
9  [ 5] [Active] [ 1] cglib (2.2)
10 [ 6] [Active] [ 1] com.ur.dashboardserver (3.3.0.SNAPSHOT)
11 [ 7] [Active] [ 1] com.ur.domain (3.3.0.SNAPSHOT)
12 ...
13 [ 56] [Active] [ 1] com.thoughtworks.xstream (1.3.1)
14 [ 57] [Active] [ 1] helloworldswing (1.0.0.SNAPSHOT)
15 ->
```

Inside the shell you can type `help` to see the list of the available commands:

```
1  -> help
2  uninstall
3  sysprop
4  bundlelevel
5  find
6  version
7  headers
8  refresh
9  start
10 obr
11 inspect
12 ps
13 stop
14 shutdown
15 help
16 update
17 install
18 log
19 cd
20 startlevel
21 resolve
22
23 Use 'help <command-name>' for more information.
24 ->
```

For example, the `headers` command can be executed to display different properties of the individual installed bundles.

A URCaps and Generated Script Code

An URCap may generate script code for installation and program nodes. To aid debugging, the generated script code for both node types is annotated with URCap information.

The following example is taken from a small program using the Hello World Swing URCap.

To see the generated script code for an URCap, it is required to save the program. Assuming the program is called `hello.urp`, the corresponding script code can be found in `hello.script`.

The script code of the installation node will be surrounded with begin/end URCap comments and information about the source of the URCap and its type:

```

1 ...
2 # begin: URCap Installation Node
3 #   Source: Hello World Swing, 1.0.0.SNAPSHOT, Universal Robots
4 #   Type: Hello World Swing
5 hello_world_swing_popup_title = "HelloWorld"
6 # end: URCap Installation Node
7 ...

```

Similarly for program nodes, script code is surrounded with begin/end URCap comments and information about the source of the URCap and its type as shown below. The program node generated by PolyScope is the first label after the `# begin: URCap Program Node`, here `$ 2`. The remaining labels until `# end : URCap Program Node`, here the statement `$ 3`, are the nodes inserted under the Hello World program node.

```

1 ...
2 # begin: URCap Program Node
3 #   Source: Hello World Swing, 1.0.0.SNAPSHOT, Universal Robots
4 #   Type: Hello World Swing
5 $ 2 "HelloWorldSwing:MyNode"
6 popup("HelloMyNode,welcome to PolyScope!", hello_world_swing_popup_title,
       False, False, blocking=True)
7 $ 3 "Wait:0.01"
8 sleep(0.01)
9 # end: URCap Program Node
10 ...

```

B My Daemon Program and Installation Node

Listing 19: Java class defining functionality for the My Daemon installation node

```

1  package com.ur.urcap.examples.mydaemonswing.impl;
2
3  import com.ur.urcap.api.contribution.DaemonContribution;
4  import com.ur.urcap.api.contribution.InstallationNodeContribution;
5  import com.ur.urcap.api.contribution.installation.CreationContext;
6  import com.ur.urcap.api.contribution.installation.InstallationAPIProvider;
7  import com.ur.urcap.api.domain.data.DataModel;
8  import com.ur.urcap.api.domain.script.ScriptWriter;
9  import com.ur.urcap.api.domain.userinteraction.inputvalidation.
    InputValidationFactory;
10 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputCallback;
11 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputFactory;
12 import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardTextInput;
13
14 import java.awt.EventQueue;
15 import java.util.concurrent.Executors;
16 import java.util.concurrent.ScheduledExecutorService;
17 import java.util.concurrent.ScheduledFuture;
18 import java.util.concurrent.TimeUnit;
19
20 public class MyDaemonInstallationNodeContribution implements
    InstallationNodeContribution {
21     private static final String POPUP_TITLE_KEY = "popuptitle";
22     private static final String XMLRPC_VARIABLE = "my_daemon_swing";
23     private static final String ENABLED_KEY = "enabled";
24     private static final String DEFAULT_VALUE = "Hello My Daemon";
25     private static final long DAEMON_TIME_OUT_NANO_SECONDS = TimeUnit.SECONDS.
        toNanos(20);
26     private static final long RETRY_TIME_TO_WAIT_MILLI_SECONDS = TimeUnit.
        SECONDS.toMillis(1);
27
28     private final MyDaemonInstallationNodeView view;
29     private final MyDaemonDaemonService daemonService;
30     private final InputValidationFactory inputValidationFactory;
31     private final DataModel model;
32     private final XmlRpcMyDaemonInterface daemonStatusMonitor;
33     private final KeyboardInputFactory keyboardInputFactory;
34     private final ScheduledExecutorService executorService = Executors.
        newScheduledThreadPool(1);
35     private ScheduledFuture<?> scheduleAtFixedRate;
36
37     public MyDaemonInstallationNodeContribution(InstallationAPIProvider
        apiProvider,
38
39         MyDaemonInstallationNodeView view,
40         DataModel model,
41         MyDaemonDaemonService daemonService,
42         XmlRpcMyDaemonInterface xmlRpcMyDaemonInterface,
43         CreationContext context) {
44         keyboardInputFactory = apiProvider.getUserInterfaceAPI().
            getUserInteraction().getKeyboardInputFactory();
45         inputValidationFactory = apiProvider.getUserInterfaceAPI().
            getUserInteraction().getInputValidationFactory();
46         this.view = view;
47         this.model = model;
48         this.daemonService = daemonService;
49         this.daemonStatusMonitor = xmlRpcMyDaemonInterface;
50         if (context.getNodeCreationType() == CreationContext.NodeCreationType.NEW)
51         {
52             model.set(POPUP_TITLE_KEY, DEFAULT_VALUE);
53         }
54         applyDesiredDaemonStatus();

```



```

53     }
54
55     @Override
56     public void openView() {
57         view.setPopupText(getPopupTitle());
58         daemonStatusMonitor.startMonitorThread();
59
60         //UI updates from non-GUI threads must use EventQueue.invokeLater (or
61         SwingUtilities.invokeLater)
62         Runnable updateUIRunnable = new Runnable() {
63             @Override
64             public void run() {
65                 EventQueue.invokeLater(new Runnable() {
66                     @Override
67                     public void run() {
68                         updateUI();
69                     }
70                 });
71             }
72         };
73         if (scheduleAtFixedRate != null) {
74             scheduleAtFixedRate.cancel(true);
75         }
76         scheduleAtFixedRate = executorService.scheduleAtFixedRate(updateUIRunnable
77             , 0, 1, TimeUnit.SECONDS);
78     }
79
80     @Override
81     public void closeView() {
82         if (scheduleAtFixedRate != null) {
83             scheduleAtFixedRate.cancel(true);
84         }
85         daemonStatusMonitor.stopMonitorThread();
86     }
87
88     @Override
89     public void generateScript(ScriptWriter writer) {
90         writer.assign(XMLRPC_VARIABLE, "rpc_factory(\"xmlrpc\", \"\");" +
91             XmlRpcMyDaemonInterface.getDaemonUrl() + "\"");
92         // Apply the settings to the daemon on program start in the Installation
93         pre-ambles
94         writer.appendLine(XMLRPC_VARIABLE + ".set_title(\"" + getPopupTitle() + "
95             \")");
96     }
97
98     private void updateUI() {
99         DaemonContribution.State state = getDaemonState();
100
101         String text = "";
102         switch (state) {
103             case RUNNING:
104                 view.setStartButtonEnabled(false);
105                 view.setStopButtonEnabled(true);
106                 text = "My Daemon runs";
107                 break;
108             case STOPPED:
109                 view.setStartButtonEnabled(true);
110                 view.setStopButtonEnabled(false);
111                 text = "My Daemon stopped";
112                 break;
113             case ERROR:
114             default:
115                 view.setStartButtonEnabled(true);
116                 view.setStopButtonEnabled(false);
117                 text = "My Daemon failed";
118                 break;
119         }
120     }

```

```

115
116     view.setStatusLabel(text);
117 }
118
119 public void onStartClick() {
120     model.set(ENABLED_KEY, true);
121     applyDesiredDaemonStatus();
122 }
123
124 public void onStopClick() {
125     model.set(ENABLED_KEY, false);
126     applyDesiredDaemonStatus();
127 }
128
129 private void applyDesiredDaemonStatus() {
130     new Thread(new Runnable() {
131         @Override
132         public void run() {
133             if (MyDaemonInstallationNodeContribution.this.isDaemonEnabled()) {
134                 // Download the daemon settings to the daemon process on initial
135                 // start for real-time preview purposes
136                 try {
137                     MyDaemonInstallationNodeContribution.this.awaitDaemonRunning();
138                     daemonStatusMonitor.setTitle(MyDaemonInstallationNodeContribution.
139                         this.getPopupTitle());
140                 } catch (Exception e) {
141                     System.err.println("Could not set the title in the daemon process.
142                         ");
143                     Thread.currentThread().interrupt();
144                 }
145             } else {
146                 daemonService.getDaemon().stop();
147             }
148         }
149     }).start();
150 }
151
152 private void awaitDaemonRunning() throws InterruptedException {
153     daemonService.getDaemon().start();
154     long endTime = System.nanoTime() + DAEMON_TIME_OUT_NANO_SECONDS;
155     while (System.nanoTime() < endTime) {
156         if (daemonStatusMonitor.isDaemonReachable()) {
157             break;
158         }
159         Thread.sleep(RETRY_TIME_TO_WAIT_MILLI_SECONDS);
160     }
161 }
162
163 public String getPopupTitle() {
164     return model.get(POPUP_TITLE_KEY, DEFAULT_VALUE);
165 }
166
167 private void setPopupTitle(String title) {
168     model.set(POPUP_TITLE_KEY, title);
169     // Apply the new setting to the daemon for real-time preview purposes
170     // Note this might influence a running program, since the actual state is
171     // stored in the daemon.
172     try {
173         daemonStatusMonitor.setTitle(title);
174     } catch (Exception e) {
175         System.err.println("Could not set the title in the daemon process.");
176     }
177 }
178
179 public KeyboardTextInput getInputForTextField() {
180     KeyboardTextInput keyboardInput = keyboardInputFactory.
181         createStringKeyboardInput();

```

```

177     keyboardInput.setErrorValidator(inputValidationFactory.
178         createStringLengthValidator(1, 255));
179     keyboardInput.setInitialValue(getPopupTitle());
180     return keyboardInput;
181 }
182 public KeyboardInputCallback<String> getCallbackForTextField() {
183     return new KeyboardInputCallback<String>() {
184         @Override
185         public void onOk(String value) {
186             setPopupTitle(value);
187             view.setPopupText(value);
188         }
189     };
190 }
191
192 public boolean isPopupTitleSet() {
193     return model.isSet(POPUP_TITLE_KEY);
194 }
195
196 private DaemonContribution.State getDaemonState() {
197     return daemonStatusMonitor.isDaemonReachable() ? daemonService.getDaemon()
198         .getState() : DaemonContribution.State.STOPPED;
199 }
200
201 private Boolean isDaemonEnabled() {
202     return model.get(ENABLED_KEY, true);
203 }
204
205 public String getXMLRPCVariable() {
206     return XMLRPC_VARIABLE;
207 }
208
209 public XmlRpcMyDaemonInterface getDaemonStatusMonitor() {
210     return daemonStatusMonitor;
211 }

```

Listing 20: Java class defining functionality for the My Daemon program node

```

1  package com.ur.urcap.examples.mydaemonswing.impl;
2
3  import com.ur.urcap.api.contribution.ProgramNodeContribution;
4  import com.ur.urcap.api.contribution.program.ProgramAPIProvider;
5  import com.ur.urcap.api.domain.data.DataModel;
6  import com.ur.urcap.api.domain.script.ScriptWriter;
7  import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputCallback;
8  import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardInputFactory;
9  import com.ur.urcap.api.domain.userinteraction.keyboard.KeyboardTextInput;
10
11 import java.awt.EventQueue;
12
13 public class MyDaemonProgramNodeContribution implements
14     ProgramNodeContribution {
15     private static final String NAME = "name";
16
17     private final ProgramAPIProvider apiProvider;
18     private final MyDaemonProgramNodeView view;
19     private final DataModel model;
20     private final XmlRpcMyDaemonInterface daemonStatusMonitor;
21     private final KeyboardInputFactory keyboardInputFactory;
22
23     public MyDaemonProgramNodeContribution(ProgramAPIProvider apiProvider,
24         MyDaemonProgramNodeView view,
25         DataModel model) {

```

```

25     keyboardInputFactory = apiProvider.getUserInterfaceAPI().
        getUserInteraction().getKeyboardInputFactory();
26     this.apiProvider = apiProvider;
27     this.view = view;
28     this.model = model;
29     this.daemonStatusMonitor = getInstallation().getDaemonStatusMonitor();
30
31 }
32
33 @Override
34 public void openView() {
35     view.setNameText(getName());
36     daemonStatusMonitor.startMonitorThread();
37
38     //UI updates from non-GUI threads must use EventQueue.invokeLater (or
        SwingUtilities.invokeLater)
39     updateUI();
40 }
41
42 private void updateUI() {
43     String title;
44     String message;
45     try {
46         // Provide a real-time preview of the daemon state
47         if (daemonStatusMonitor.isDaemonReachable()) {
48             title = daemonStatusMonitor.getTitle();
49             message = daemonStatusMonitor.getMessage(getName());
50         } else {
51             title = message = "<Daemon disconnected>";
52         }
53     } catch (Exception e) {
54         System.err.println("Could not retrieve essential data from the daemon
            process for the preview.");
55         title = message = "<Daemon disconnected>";
56     }
57     setTitleAndMessage(title, message);
58 }
59
60 private void setTitleAndMessage(final String title, final String message) {
61     EventQueue.invokeLater(new Runnable() {
62         @Override
63         public void run() {
64             MyDaemonProgramNodeContribution.this.updatePreview(title, message);
65         }
66     });
67 }
68
69 private void updatePreview(String title, String message) {
70     view.setTitlePreview(title);
71     view.setMessagePreview(message);
72     updateUI();
73 }
74
75 @Override
76 public void closeView() {
77     daemonStatusMonitor.stopMonitorThread();
78 }
79
80 @Override
81 public String getTitle() {
82     return "My Daemon: " + getName();
83 }
84
85 @Override
86 public boolean isDefined() {
87     return daemonStatusMonitor.isDaemonReachable() && !getName().isEmpty() &&
        getInstallation().isPopupTitleSet();

```

```

88     }
89
90     @Override
91     public void generateScript(ScriptWriter writer) {
92         // Interact with the daemon process through XML-RPC calls
93         // Note, alternatively plain sockets can be used.
94         writer.assign("mydaemon_message", getInstallation().getXMLRPCVariable() +
95             ".get_message(\"" + getName() + "\")");
96         writer.assign("mydaemon_title", getInstallation().getXMLRPCVariable() + ".
97             get_title()");
98         writer.appendLine("popup(mydaemon_message, mydaemon_title, False, False,
99             blocking=True)");
100        writer.writeChildren();
101    }
102
103    public KeyboardTextInput getInputForTextField() {
104        KeyboardTextInput keyboardTextInput = keyboardInputFactory.
105            createStringKeyboardInput();
106        keyboardTextInput.setInitialValue(getName());
107        return keyboardTextInput;
108    }
109
110    public KeyboardInputCallback<String> getCallbackForTextField() {
111        return new KeyboardInputCallback<String>() {
112            @Override
113            public void onOk(String value) {
114                setName(value);
115                view.setNameText(value);
116            }
117        };
118    }
119
120    private String getName() {
121        return model.get(NAME, "");
122    }
123
124    private void setName(String name) {
125        if (!name.equals(name)) {
126            model.remove(NAME);
127        } else {
128            model.set(NAME, name);
129        }
130    }
131
132    private MyDaemonInstallationNodeContribution getInstallation() {
133        return apiProvider.getProgramAPI().getInstallationNode(
134            MyDaemonInstallationNodeContribution.class);
135    }
136 }

```

Listing 21: Java class for XML-RPC communication

```

1  package com.ur.urcap.examples.mydaemonswing.impl;
2
3  import org.apache.xmlrpc.XmlRpcException;
4  import org.apache.xmlrpc.client.XmlRpcClient;
5  import org.apache.xmlrpc.client.XmlRpcClientConfigImpl;
6
7  import java.net.MalformedURLException;
8  import java.net.URL;
9  import java.util.ArrayList;
10 import java.util.Collections;
11 import java.util.concurrent.Executors;
12 import java.util.concurrent.ScheduledExecutorService;
13 import java.util.concurrent.ScheduledFuture;
14 import java.util.concurrent.TimeUnit;

```

```

15 import java.util.concurrent.atomic.AtomicBoolean;
16
17 public class XmlRpcMyDaemonInterface {
18     private static final int PORT = 40405;
19     private static final String HOST_IP = "127.0.0.1";
20     private static final XmlRpcClient XML_RPC_CLIENT = new XmlRpcClient();
21     private static final XmlRpcClientConfigImpl XML_RPC_CLIENT_CONFIG = new
        XmlRpcClientConfigImpl();
22
23     private final AtomicBoolean isDaemonReachable = new AtomicBoolean(false);
24     private final ScheduledExecutorService executorService = Executors.
        newScheduledThreadPool(1);
25     private ScheduledFuture<?> scheduleAtFixedRate;
26
27     public XmlRpcMyDaemonInterface() {
28         setupXmlRpcClient();
29         startMonitorThread();
30     }
31
32     public static String getDaemonUrl() {
33         return "http://" + HOST_IP + ":" + PORT + "/RPC2";
34     }
35
36     private static void setupXmlRpcClient() {
37         try {
38             XML_RPC_CLIENT_CONFIG.setEnabledForExtensions(true);
39             XML_RPC_CLIENT_CONFIG.setServerURL(new URL(getDaemonUrl()));
40             XML_RPC_CLIENT_CONFIG.setConnectionTimeout(1000); //1s
41             XML_RPC_CLIENT.setConfig(XML_RPC_CLIENT_CONFIG);
42         } catch (MalformedURLException e) {
43             e.printStackTrace();
44         }
45     }
46
47     public void startMonitorThread() {
48         Runnable containerMonitorRunnable = new Runnable() {
49             @Override
50             public void run() {
51                 isDaemonReachable.set(XmlRpcMyDaemonInterface.this.
                    tryExecuteIsReachable());
52             }
53         };
54
55         stopMonitorThread();
56         scheduleAtFixedRate = executorService.scheduleWithFixedDelay(
            containerMonitorRunnable, 0, 1, TimeUnit.SECONDS);
57     }
58
59     private boolean tryExecuteIsReachable() {
60         try {
61             return (Boolean) XML_RPC_CLIENT.execute("isReachable", new ArrayList<
                String>());
62         } catch (XmlRpcException ignored) {
63             return false;
64         }
65     }
66
67     public void stopMonitorThread() {
68         if (scheduleAtFixedRate != null) {
69             scheduleAtFixedRate.cancel(true);
70         }
71     }
72
73     public boolean isDaemonReachable() {
74         return isDaemonReachable.get();
75     }
76

```

```
77     public String getTitle() throws XmlRpcException, UnknownResponseException {
78         return processString(XML_RPC_CLIENT.execute("get_title", new ArrayList<
              String>()));
79     }
80
81     public void setTitle(String title) throws XmlRpcException,
            UnknownResponseException {
82         processString(XML_RPC_CLIENT.execute("set_title", Collections.
            singletonList(title)));
83     }
84
85     public String getMessage(String name) throws XmlRpcException,
            UnknownResponseException {
86         return processString(XML_RPC_CLIENT.execute("get_message", Collections.
            singletonList(name)));
87     }
88 }
89
90 private String processString(Object response) throws
            UnknownResponseException {
91     if (response instanceof String) {
92         return (String) response;
93     } else {
94         throw new UnknownResponseException();
95     }
96 }
97 }
```

Listing 22: hello-world.py Python 2.5 daemon example

```

1  #!/usr/bin/env python
2
3  import sys
4
5  from SimpleXMLRPCServer import SimpleXMLRPCServer
6  from SocketServer import ThreadingMixIn
7
8  title = ""
9
10 def isReachable():
11     return True
12
13 def set_title(new_title):
14     global title
15     title = new_title
16     return title
17
18 def get_title():
19     tmp = ""
20     if str(title):
21         tmp = title
22     else:
23         tmp = "No_title_set"
24     return tmp + "_Python)"
25
26 def get_message(name):
27     if str(name):
28         return "Hello_" + str(name) + ",_welcome_to_PolyScope!"
29     else:
30         return "No_name_set"
31
32 sys.stdout.write("MyDaemon_daemon_started")
33 sys.stderr.write("MyDaemon_daemon_started")
34
35 class MultithreadedSimpleXMLRPCServer(ThreadingMixIn, SimpleXMLRPCServer):
36     pass
37
38 server = MultithreadedSimpleXMLRPCServer(("127.0.0.1", 40405))
39 server.register_function(isReachable, "isReachable")
40 server.RequestHandlerClass.protocol_version = "HTTP/1.1"
41 server.register_function(set_title, "set_title")
42 server.register_function(get_title, "get_title")
43 server.register_function(get_message, "get_message")
44 server.serve_forever()

```